

README

CP 版本说明

本说明用于维护：每套试卷均提供 C 语言版与 Python 版。两版题目、分值、说明与非代码答案应保持一致；只允许程序代码、代码填空答案及代码语言相关记号不同。

1. 本目录中的文件按 `-C.tex` 与 `-P.tex` 成对出现。若后续修订某一版本的题面、选择题或非代码解答，需要同步修改另一版本；若只修订程序实现或代码填空，则应在另一版本中作等价语言转换。

数据结构与算法 A

14 秋-期末

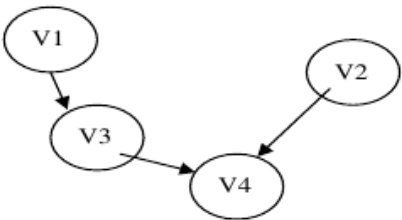
题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、填空（27 分）

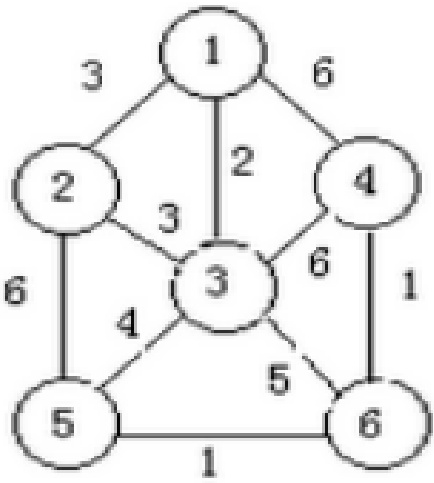
1. （2 分）当各边上的权值 ____ 时，BFS 算法可用来解决单源最短路径问题。

A. 均相等 B. 均互不相等 C. 不一定相等

2. （4 分）有向图 G 如下图所示：



- (1) 写出所有可能的拓扑序列：_____。
- (2) 添加一条弧 _____ 之后，则仅有唯一的拓扑序列。
3. （2 分）设有如下所示无向图 G，用 Prim 算法构造最小生成树所走过的边



的集合 $E=\{(1, 3), (1, 2), (3, 5), (5, 6), (6, 4)\}$ （用边 $(2, 3)$ 代替 $(1, 2)$ 也可以）。

4. （2 分）如果只想得到 1000 个元素组成的序列中第 5 个最小元素之前的部分排序的序列，用 ____ 方法最快。

A. 起泡排序 B. 快速排列 C. 堆排序 D. 简单选择排序

5. （2 分）若要求尽可能快地对序列进行稳定的排序，则应选 ____

A. 快速排序 B. 归并排序 C. 冒泡排序 D. 归并排序

V1

V3

V4

V2

6. （2 分）在文件局部有序或文件长度较小的情况下，最佳的排序方法是 ____。

A. 直接插入排序 B. 冒泡排序 C. 直接选择排序 D. 归并排序

7. (2 分) 设输入的关键字满足 $K_1 > K_2 > \dots > K_n$ ，缓冲区大小为 m ，用置换选择排序方法可产生 _____ 个初始归并段。
8. (2 分) 设有 13 个初始归并段，其长度分别为 28, 16, 37, 42, 5, 9, 13, 14, 20, 17, 30, 12, 18。试计算最佳 4 路归并树的带权路径长度 $WPL =$ _____。
9. (3 分) 设有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908，则对其进行折半查找时，等概率下搜索成功的平均查找长度和不成功的平均查找长度分别为 _____ 和 _____。
10. (2 分) 设散列表长度为 14 (地址下标为 $0 \dots 13$)，哈希函数为 $H(key) = key \% 11$ ，表中已有数据的关键字为 15, 38, 61, 84 共四个，现要将关键字为 49 的结点将加入表中，用二次探查法解决冲突，则放入的位置是 _____。
- A. 8 B. 3 C. 5 D. 9
11. (2 分) 在一棵含有 1023 个关键字的 4 阶 B 树中进行查找 (不读取数据主文件)，至多读盘 _____ 次。
- A. 7 B. 8 C. 10 D. 9
12. (2 分) 在一棵阶为 k 的红黑树中，内部结点最少有 $2k-1$ 个；从根到叶的简单路径长度的范围为 $[k, 2k]$ 。

二、辨析与简答 (30 分)

1.

(8 分) 将关键字序列 (7、8、30、11、18、9、14) 散列存储到散列表中。

散列表是一个下标从 0 开始的一维数组，散列函数为： $H(key) = (key * 3) \text{ MOD } 7$ ，处理冲突采用线性探测法，要求装填 (载) 因子为 0.7。

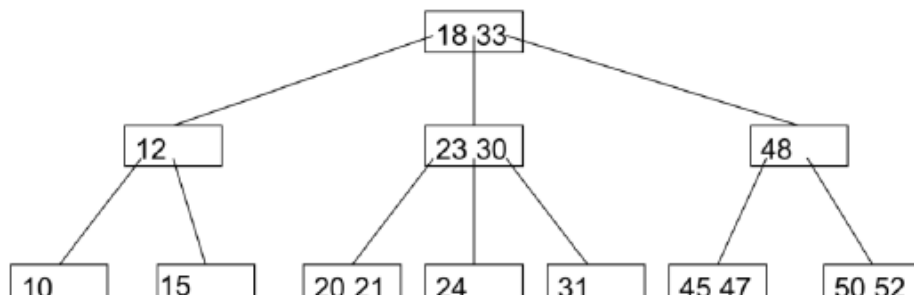
7, 处理冲突采用线性探测法，要求装填 (载) 因子为 0.7。

(1) 请画出所构造的散列表。

(2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

2.

(6 分) 设有如下一棵 3 阶 B 树，请画出在其中插入关键码 19 后的 B 树，并给出



插入过程中的访外 (读写) 次数；在此基础上再删除关键码 48，请画出删除后的 B 树及删除过程中的访外 (读写) 次数。

3. (6 分) 将下列字符序列 E A S Y Q U E S T I O N 依次插入到初始为空的红黑树 (RB-tree) 中，请画出最终得到的红黑树。(用圆圈表示红色结点，方框表示黑色结点，外部空叶结点可省略不画)。

4. (4 分) 利用广义表的 Head 和 Tail 运算，把原子 d 分别从下列广义表中分离出来， $L_1 = (((((a), b), d), e))$; $L_2 = (a, (b, ((d)), e))$ 。

5. (6 分) 将关键码 1, 2, 3, ..., (2k-1) 依次插入到一棵初始为空的 AVL 树中，试证明结果是一棵高度为 k 的完全满二叉树。

三、算法填空 (18 分)

阅读和完成下面的代码 (每空可不限一条语句)。

1.

(10 分) 以单向链表为存储结构，实现选择排序算法 (得到非递减序列)。

```
struct node {
    int key;
    node *next;
```

```
};
node* sort_linked_list(node* head) {
i = head;
while (i!=NULL) {
    填空 1 _____;
    while (j!=NULL) {
        if (填空 2 _____) 填空 3 _____;
        填空 4 _____;
    }
    填空 5 _____;
}
return head;
}
```

2.

(8 分) 下面是败者树在内部结点从右分支向上比赛的成员函数实现, 请将空缺部分补充完整。

```
template<class T>
void LoserTree<T>::Play(int p, int lc, int rc, int(*winner)(T A[], int b, int c),
int(*loser)(T A[], int b, int c)) {
    B[p] = loser(L, lc, rc);
    int temp1, temp2;
    (1) _____;
    while (p>1 && (2) _____) {
        temp2 = winner(L, temp1, B[p/2]);
        (3) _____;
        temp1 = temp2;
        p/=2;
    }
    (4) _____;
}
```

3. (8 分) 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下:

```
Class Graph {
public:
    int VerticesNum();
    int EdgesNum();
    Edge FirstEdge(int oneVertex);
    Edge NextEdge(Edge preEdge);
    bool IsEdge(Edge onEdge);
    int FromVertex(Edge oneEdge);
    int ToVertex(Edge oneEdge);
};
int visited[MAXSIZE];
// 初始化为 0
int GetPathNum_Len(Graph& G, int i, int j, int len) {
    if (填空 1 _____) return 1;
    sum = 0;
    // sum 表示通过本结点的路径数
    visited[i] = 1;
    for (Edge e = G.FirstEdge(i); G.IsEdge(e); e = G.NextEdge(e)) {
        int v = G.ToVertex(e);
```

```

if (!visited[v])
    填空 2 _____
} // for
visited[i] = 0;
//本题允许曾经被访问过的结点出现在另一条路径中
return sum;
} // GetPathNum_Len

```

四、算法设计与分析（25 分）

请尽量按照试题中的要求来写高效率的可读算法。应该申明算法思想，在代码种加以恰当的注释。

1.

伸展树是一种自平衡的 BST 树，它可以通过旋转来实现树结构的动态调整。

伸展树结点的数据结构定义如下：

```

struct TreeNode{
int data;
TreeNode * father,* left,* right;
};

```

其成员函数包括：

1) Delete zig(TreeNode* x); 其功能是删除以 x 结点为根的子树；

2) Splay(TreeNode* x , TreeNode* f); 其功能是将 x 旋转为 f 的子结点（f 是 x 的祖先）。特殊的，把 x 旋转到根结点即 Splay(x, NULL)。

请运用上述给定的成员函数，实现删除树 rt 中所有大于 u 小于 v 的结点（假设 u, v 是 Splay 树中的结点，且树种所有的节点值不重复）。

函数原型为：void DeleteSubTree(TreeNode* rt, TreeNode* u, TreeNode* v)

解答：把 u 结点旋转到根；把 v 旋转为 u 的右儿子；删除 v 结点的左子树前面几句各 3 分，最后一句 1 分。如果思路、注释各 1 分，不写则扣。

```

void DeleteUV(TreeNode* rt, TreeNode* u, TreeNode* v ) {
Splay(u, NULL);
Splay(v, u);
Delete(v->lchild);
v->lchild = NULL;
}

```

数据结构与算法 A

14 秋-期末

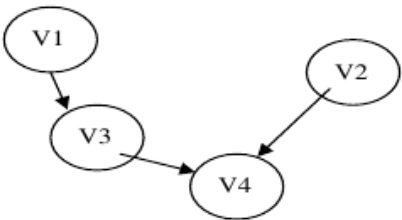
题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、填空（27 分）

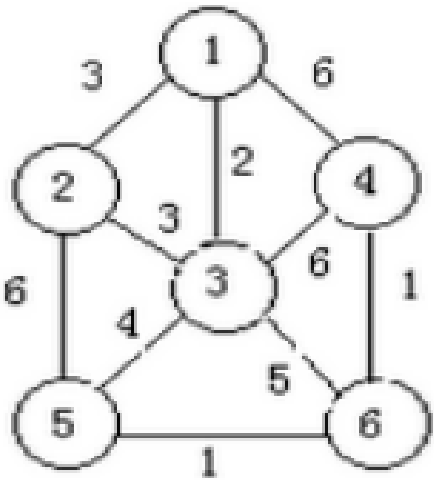
1. （2 分）当各边上的权值 ____ 时，BFS 算法可用来解决单源最短路径问题。

A. 均相等 B. 均互不相等 C. 不一定相等

2. （4 分）有向图 G 如下图所示：



- (1) 写出所有可能的拓扑序列：_____。
- (2) 添加一条弧 _____ 之后，则仅有唯一的拓扑序列。
3. （2 分）设有如下所示无向图 G，用 Prim 算法构造最小生成树所走过的边



的集合 $E=\{(1, 3), (1, 2), (3, 5), (5, 6), (6, 4)\}$ （用边 $(2, 3)$ 代替 $(1, 2)$ 也可以）。

4. （2 分）如果只想得到 1000 个元素组成的序列中第 5 个最小元素之前的部分排序的序列，用 ____ 方法最快。

A. 起泡排序 B. 快速排列 C. 堆排序 D. 简单选择排序

5. （2 分）若要求尽可能快地对序列进行稳定的排序，则应选 ____

A. 快速排序 B. 归并排序 C. 冒泡排序 D. 归并排序

V1

V3

V4

V2

6. （2 分）在文件局部有序或文件长度较小的情况下，最佳的排序方法是 ____。

A. 直接插入排序 B. 冒泡排序 C. 直接选择排序 D. 归并排序

7. (2 分) 设输入的关键字满足 $K_1 > K_2 > \dots > K_n$ ，缓冲区大小为 m ，用置换选择排序方法可产生 _____ 个初始归并段。
8. (2 分) 设有 13 个初始归并段，其长度分别为 28, 16, 37, 42, 5, 9, 13, 14, 20, 17, 30, 12, 18。试计算最佳 4 路归并树的带权路径长度 $WPL =$ _____。
9. (3 分) 设有序顺序表中的元素依次为 017, 094, 154, 170, 275, 503, 509, 512, 553, 612, 677, 765, 897, 908，则对其进行折半查找时，等概率下搜索成功的平均查找长度和不成功的平均查找长度分别为 _____ 和 _____。
10. (2 分) 设散列表长度为 14 (地址下标为 $0 \dots 13$)，哈希函数为 $H(key) = key \% 11$ ，表中已有数据的关键字为 15, 38, 61, 84 共四个，现要将关键字为 49 的结点将加入表中，用二次探查法解决冲突，则放入的位置是 _____。
- A. 8 B. 3 C. 5 D. 9
11. (2 分) 在一棵含有 1023 个关键字的 4 阶 B 树中进行查找 (不读取数据主文件)，至多读盘 _____ 次。
- A. 7 B. 8 C. 10 D. 9
12. (2 分) 在一棵阶为 k 的红黑树中，内部结点最少有 $2k-1$ 个；从根到叶的简单路径长度的范围为 $[k, 2k]$ 。

二、辨析与简答 (30 分)

1.

(8 分) 将关键字序列 (7、8、30、11、18、9、14) 散列存储到散列表中。

散列表是一个下标从 0 开始的一维数组，散列函数为： $H(key) = (key * 3) \text{ MOD } 7$ ，处理冲突采用线性探测法，要求装填 (载) 因子为 0.7。

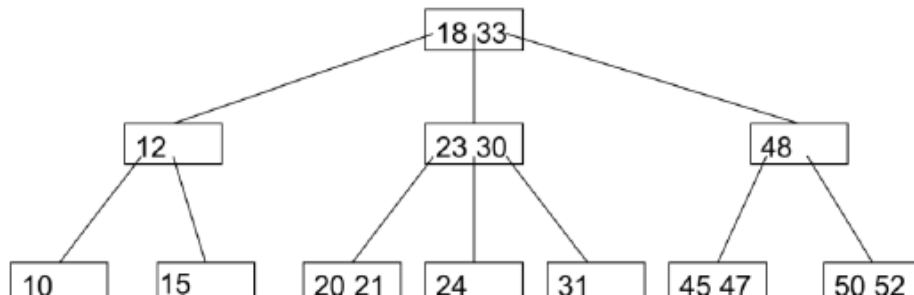
7, 处理冲突采用线性探测法，要求装填 (载) 因子为 0.7。

(1) 请画出所构造的散列表。

(2) 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

2.

(6 分) 设有如下一棵 3 阶 B 树，请画出在其中插入关键码 19 后的 B 树，并给出



插入过程中的访外 (读写) 次数；在此基础上再删除关键码 48，请画出删除后的 B 树及删除过程中的访外 (读写) 次数。

3. (6 分) 将下列字符序列 E A S Y Q U E S T I O N 依次插入到初始为空的红黑树 (RB-tree) 中，请画出最终得到的红黑树。(用圆圈表示红色结点，方框表示黑色结点，外部空叶结点可省略不画)。

4. (4 分) 利用广义表的 Head 和 Tail 运算，把原子 d 分别从下列广义表中分离出来， $L_1 = (((((a), b), d), e))$; $L_2 = (a, (b, ((d)), e))$ 。

5. (6 分) 将关键码 1, 2, 3, ..., (2k-1) 依次插入到一棵初始为空的 AVL 树中，试证明结果是一棵高度为 k 的完全满二叉树。

三、算法填空 (18 分)

阅读和完成下面的代码 (每空可不限一条语句)。

1.

(10 分) 以单向链表为存储结构，实现选择排序算法 (得到非递减序列)。

```
class node {
int key;
node *next;
```

```
};
node* sort_linked_list(node* head) {
i = head;
while (i!=None) {
    填空 1 _____;
    while (j!=None) {
        if (填空 2 _____) 填空 3 _____;
        填空 4 _____;
    }
    填空 5 _____;
}
return head;
}
```

2.

(8 分) 下面是败者树在内部结点从右分支向上比赛的成员函数实现, 请将空缺部分补充完整。

```
template<class T>
def LoserTree<T>::Play(int p, int lc, int rc, int(*winner)(T A[], int b, int c),
int(*loser)(T A[], int b, int c)) {
    B[p] = loser(L, lc, rc);
    int temp1, temp2;
    (1) _____;
    while p > 1 and p temp2 = winner(L, temp1, B[p/2]);
    (3) _____;
    temp1 = temp2;
    p/=2;
}
(4) _____;
}
```

3. (8 分) 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下:

```
Class Graph {
public:
    int VerticesNum();
    int EdgesNum();
    Edge FirstEdge(int oneVertex);
    Edge NextEdge(Edge preEdge);
    IsEdge(Edge onEdge);
    int FromVertex(Edge oneEdge);
    int ToVertex(Edge oneEdge);
};
int visited[MAXSIZE];
// 初始化为 0
int GetPathNum_Len(G, int i, int j, int len) {
    if (填空 1 _____) return 1;
    sum = 0;
    // sum 表示通过本结点的路径数
    visited[i] = 1;
    for (Edge e = G.FirstEdge(i); G.IsEdge(e); e = G.NextEdge(e)) {
        int v = G.ToVertex(e);
        if (not visited[v])
```


填空 2 _____

```
// for
visited[i] = 0;
//本题允许曾经被访问过的结点出现在另一条路径中
return sum;
} // GetPathNum_Len
```

四、算法设计与分析（25 分）

请尽量按照试题中的要求来写高效率的可读算法。应该申明算法思想，在代码种加以恰当的注释。

1.

伸展树是一种自平衡的 BST 树，它可以通过旋转来实现树结构的动态调整。

伸展树结点的数据结构定义如下：

```
class TreeNode{
int data;
TreeNode * father,* left,* right;
};
```

其成员函数包括：

1) Delete zig(TreeNode* x); 其功能是删除以 x 结点为根的子树；

2) Splay(TreeNode* x , TreeNode* f); 其功能是将 x 旋转为 f 的子结点（f 是 x 的祖先）。特殊的，把 x 旋转到根结点即 Splay(x, None)。

请运用上述给定的成员函数，实现删除树 rt 中所有大于 u 小于 v 的结点（假设 u, v 是 Splay 树中的结点，且树种所有的节点值不重复）。

函数原型为：def DeleteSubTree(TreeNode* rt, TreeNode* u, TreeNode* v)

解答：把 u 结点旋转到根；把 v 旋转为 u 的右儿子；删除 v 结点的左子树前面几句各 3 分，最后一句 1 分。如果思路、注释各 1 分，不写则扣。

```
def DeleteUV(TreeNode* rt, TreeNode* u, TreeNode* v ) {
Splay(u, None);
Splay(v, u);
Delete(v.lchild);
v.lchild = None;
}
```

数据结构与算法 A

15 秋-期末

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择题（每题 2 分，共 20 分）

1.

若要一个运行时间为 $100n^2$ 的算法在相同机器上快于另一个运行时间为 $2n$ 的算法，则最小的 n 值应为 _____。

2.

某算法仅含顺序执行的语句 1 和语句 2，语句 1 执行次数为 $20n^2+3n\log n$ ，而语句 2 执行次数为 $2n^3$ ，则该算法的时间复杂度为 _____。

3.

下面术语中，_____与数据结构的存储结构无关。

A. 顺序表 B. 链表 C. 队列 D. 循环链表 E. 堆

4.

在一个具有 n 个结点的有序单链表中插入一个新结点并仍保持其有序的时间复杂度为 _____。

A. $O(n \log_2 n)$ B. $O(1)$ C. $O(n)$

D. $O(n^2)$

5.

若元素 a、b、c、d、e、f 依次进栈，允许进栈、退栈操作交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是 _____。（ ）

A. dcebf a

B. cbdaef

C. bcaefd

D. afedcb

6.

表达式 $3 * 2^{(4+2*2-6*3)} - 5$ 的求值过程中，当扫描到 6 时，操作数栈和运算符栈为，其中 ^ 为乘幂。（ ）

A. 3,2,4,1,1; (*^(+*-

B. 3,2,8; (*^-

C. 3,2,4,2,2; (*^(-

D. 3,2,8; (*^(-

7.

有 4 棵结点存储整数关键码的二叉树，若它们的中序遍历序列分别为：A. 5, 3, 1, 2, 4

B. 1, 2, 3, 4, 5 C. 5, 4, 3, 2, 1 D. 1, 3, 5, 4, 2。请问其中可能是二叉搜索树的包括 _____。

8.

（4 分）一棵 Huffman 树的带权外部路径长度定义为所有叶结点权值与其深度的乘积之和。

设根结点的深度为 0，对集合 { 2, 3, 5, 7, 8 } 构造二叉 Huffman 树，则其最小带权外部路径长度为 _____。

9.

两个字符串相等的充分必要条件是 _____。

（两字

符串的长度相等且两字符串中对应位置的字符也相等）

10. 2-3 树是一种满足以下两个条件的特殊的树：（1）每个内部结点有两个或三个子结点；（2）

所有叶结点到根的路径长度相同。如果一棵 2-3 树有 9 个叶结点，那么它可能有 _____ 个非叶结点。

二、辨析与简答题（共 36 分）

1.

(6 分) 什么是循环队列? 循环队列的优点是什么? 如何判别它的空和满?

2.

(6 分) 在包含 n 个结点的单链表、双链表和单循环链表中, 若仅知道指针 p 指向某结点, 不知道表的头指针, 能否将结点 p 从相应的链表中删去? 若可以, 其时间复杂度各为多少?

3.

(8 分) 请分别计算模式 $p = \text{"aabaac"}$ 优化前和优化后的 next 数组, 并按照优化后的 next 数组针对目标 $t = \text{"aabaabaabaac"}$ 进行 KMP 快速模式匹配, 请画出匹配过程的示意图并计算匹配过程中的比较次数。

4.

(8 分) 根据树的双标记层次遍历序列, 求其带度数的后根遍历序列。譬如, 已知一棵树的双标记层次遍历序列如下:

A : ltag: 0 , rtag 1

B : ltag: 0 , rtag 0

C : ltag: 0 , rtag 1

D : ltag: 1 , rtag 0

G : ltag: 0 , rtag 1

E : ltag: 0 , rtag 1

H : ltag: 1 , rtag 1

F : ltag: 1 , rtag 0

I : ltag: 1 , rtag 1

则其带度数的后根遍历序列为: D0 H0 G1 B2 F0 I0 E2 C1 A2 (注: 各个结点按照“结点名度数”的方式给出, 结点之间用空格分隔)。现某棵树的双标记层次遍历序列如下:

A : ltag: 0 , rtag 1

B : ltag: 0 , rtag 0

G : ltag: 1 , rtag 1

C : ltag: 0 , rtag 0

F : ltag: 0 , rtag 1

D : ltag: 1 , rtag 0

E : ltag: 1 , rtag 1

H : ltag: 1 , rtag 0

I : ltag: 1 , rtag 1

请给出其带度数的后根遍历序列。

5.

(8 分) 使用重量权衡合并规则(权重合并规则)与路径压缩, 对下列从 0 到 15 之间的数的等价对进行归并。在初始情况下, 集合中的每个元素分别在独立的等价类中。当两棵树规模同样大时, 使结点数值较大的根结点作为值较小的根结点的子结点。

(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9) (4,14) (6,7) (8,10) (8,7) (7,0) (10,15) (10,13)
请填写下面表格的空白部分树的父指针表示法的数组表示。也就是所有等价对都被处理之后, 所得父结点的下标值(没有父结点则填“-1”)。

父结点下标

结点值

0 1

2 3

4 5

7 8

9 10 11 12 13 14 15

结点的下标 0 1

2 3

4 5

7 8

9 10 11 12 13 14 15

三、算法填空题（每题 12 分，共 24 分）

1.

（每空 3 分，共 12 分）下面的算法利用一个栈将一给定的序列从小到大排序。长度为 n 的序列由 $1, 2, \dots, n$ 这 n 个连续的正整数组成。请补全下面的代码段，使其可以判断给定的序列是否可以利用一个栈进行排序，如果可以，输出相应的操作。

例如：序列 4 3 1 2，此序列可以排序，输出：push push push pop push pop pop pop。

template <class T> // 栈的元素类型为 T

```
class Stack {
```

```
public:
```

```
void clear();
```

```
// 变为空栈
```

```
void push(const T item);
```

```
// item 入栈
```

```
T pop();
```

```
// 返回栈顶内容并弹出
```

```
T top();
```

```
// 返回栈顶内容但不弹出
```

```
bool isEmpty();
```

```
// 若栈已空返回真
```

```
bool isFull();
```

```
// 若栈已满返回真
```

```
};
```

```
Stack<int> s;
```

```
// s 为栈
```

```
int sequence[maxLen], len; // sequence 为待排序序列，len 为序列长度
```

```
bool islegal() {
```

```
// 判断序列是否可以排序
```

```
int i, j, k;
```

```
for (i = 0; i < len; i++) {
```

```
for (j = i+1; j < len; j++) {
```

```
for (k = j+1; k < len; k++) {
```

```
if ( )
```

```
return false;
```

```
}
```

```
}
```

```
}
```

```
return true;
```

```
}
```

```
void transform() {
```

```
// 输出转换操作
```

```
int cur = 1, i = 0;
```

```
while ( ) {
```

```
while (s.isEmpty() || ) {
```

```
cout << "push ";
```

```
}
```

```
s.pop();
```

```
cout << "pop ";
```

```
++i;
```

```
}
```

```
}
```

2.

(每空 3 分, 分析 3 分, 共 12 分) 以下算法用来判断两棵树是否相同, 试补全下面的代码段, 并分析算法的运行时间代价。

```
template <class T>
bool IsEqual(TreeNode<T> *root1, TreeNode<T> *root2) {
while (root1 != NULL && root2 != NULL) {
if (root1->value() != root2->value())
return _____ ;
if (_____)
return false;
root1 = root1->RightSibling();
root2 = root2->RightSibling();
}
if (_____)
return true;
return false;
}
```

四、

分析证明题 (共 20 分)

1.

(5 分) 请证明非空满 K 叉树的叶结点数目为 $(K-1)*n+1$, 其中 n 为分支结点数目。

2.

(15 分) 有 n 个数 $K_1, K_2, \dots, K_n$ 顺序存储在数组中, 形成一棵 n 个结点的完全二叉树。现在要将它按如下方式建成一个最小值堆: 从左至右扫描整个数组, 将第 i 个元素 K_i ($i = 1, 2, \dots, n$) 与其父结点 $K[\lfloor i/2 \rfloor]$ 比较, 如果 $K_i \geq K[\lfloor i/2 \rfloor]$, 则不再进行操作; 若 $K_i < K[\lfloor i/2 \rfloor]$, 将 2 个结点对换, 再将 $K[\lfloor i/2 \rfloor]$ 与 $K[\lfloor i/4 \rfloor]$ 比较, 以此类推, 直到比较到根结点 K_1 。

(1) 按照这种建堆方式, 最坏情况下建堆的时间复杂度是多少?

(2) 教材上使用的“筛选法”建堆 (Sift down), 最坏情况下建堆的时间复杂度是多少?

(3) 上述 2 种建堆法有类似之处, 请问它们的时间复杂度是否一样, 若不同, 则请说明导致它们时间复杂度不同的原因。

数据结构与算法 A

15 秋-期末

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择题（每题 2 分，共 20 分）

1.

若要一个运行时间为 $100n^2$ 的算法在相同机器上快于另一个运行时间为 $2n$ 的算法，则最小的 n 值应为 _____。

2.

某算法仅含顺序执行的语句 1 和语句 2，语句 1 执行次数为 $20n^2+3n\log n$ ，而语句 2 执行次数为 $2n^3$ ，则该算法的时间复杂度为 _____。

3.

下面术语中，_____与数据结构的存储结构无关。

A. 顺序表 B. 链表 C. 队列 D. 循环链表 E. 堆

4.

在一个具有 n 个结点的有序单链表中插入一个新结点并仍保持其有序的时间复杂度为 _____。

A. $O(n \log_2 n)$ B. $O(1)$ C. $O(n)$

D. $O(n^2)$

5.

若元素 a、b、c、d、e、f 依次进栈，允许进栈、退栈操作交替进行，但不允许连续三次进行退栈操作，则不可能得到的出栈序列是 _____。（ ）

A. dcebf a

B. cbdaef

C. bcaefd

D. afedcb

6.

表达式 $3 * 2^{(4+2*2-6*3)} - 5$ 的求值过程中，当扫描到 6 时，操作数栈和运算符栈为，其中 ^ 为乘幂。（ ）

A. 3,2,4,1,1; (*^(+*-

B. 3,2,8; (*^-

C. 3,2,4,2,2; (*^(-

D. 3,2,8; (*^(-

7.

有 4 棵结点存储整数关键码的二叉树，若它们的中序遍历序列分别为：A. 5, 3, 1, 2, 4

B. 1, 2, 3, 4, 5 C. 5, 4, 3, 2, 1 D. 1, 3, 5, 4, 2。请问其中可能是二叉搜索树的包括 _____。

8.

（4 分）一棵 Huffman 树的带权外部路径长度定义为所有叶结点权值与其深度的乘积之和。

设根结点的深度为 0，对集合 { 2, 3, 5, 7, 8 } 构造二叉 Huffman 树，则其最小带权外部路径长度为 _____。

9.

两个字符串相等的充分必要条件是 _____。

（两字

符串的长度相等且两字符串中对应位置的字符也相等）

10. 2-3 树是一种满足以下两个条件的特殊的树：（1）每个内部结点有两个或三个子结点；（2）

所有叶结点到根的路径长度相同。如果一棵 2-3 树有 9 个叶结点，那么它可能有 _____ 个非叶结点。

二、辨析与简答题（共 36 分）

1.

(6 分) 什么是循环队列? 循环队列的优点是什么? 如何判别它的空和满?

2.

(6 分) 在包含 n 个结点的单链表、双链表和单循环链表中, 若仅知道指针 p 指向某结点, 不知道表的头指针, 能否将结点 p 从相应的链表中删去? 若可以, 其时间复杂度各为多少?

3.

(8 分) 请分别计算模式 $p = \text{"aabaac"}$ 优化前和优化后的 next 数组, 并按照优化后的 next 数组针对目标 $t = \text{"aabaabaabaac"}$ 进行 KMP 快速模式匹配, 请画出匹配过程的示意图并计算匹配过程中的比较次数。

4.

(8 分) 根据树的双标记层次遍历序列, 求其带度数的后根遍历序列。譬如, 已知一棵树的双标记层次遍历序列如下:

A : ltag: 0 , rtag 1

B : ltag: 0 , rtag 0

C : ltag: 0 , rtag 1

D : ltag: 1 , rtag 0

G : ltag: 0 , rtag 1

E : ltag: 0 , rtag 1

H : ltag: 1 , rtag 1

F : ltag: 1 , rtag 0

I : ltag: 1 , rtag 1

则其带度数的后根遍历序列为: D0 H0 G1 B2 F0 I0 E2 C1 A2 (注: 各个结点按照“结点名度数”的方式给出, 结点之间用空格分隔)。现某棵树的双标记层次遍历序列如下:

A : ltag: 0 , rtag 1

B : ltag: 0 , rtag 0

G : ltag: 1 , rtag 1

C : ltag: 0 , rtag 0

F : ltag: 0 , rtag 1

D : ltag: 1 , rtag 0

E : ltag: 1 , rtag 1

H : ltag: 1 , rtag 0

I : ltag: 1 , rtag 1

请给出其带度数的后根遍历序列。

5.

(8 分) 使用重量权衡合并规则(权重合并规则)与路径压缩, 对下列从 0 到 15 之间的数的等价对进行归并。在初始情况下, 集合中的每个元素分别在独立的等价类中。当两棵树规模同样大时, 使结点数值较大的根结点作为值较小的根结点的子结点。

(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9) (4,14) (6,7) (8,10) (8,7) (7,0) (10,15) (10,13)

请填写下面表格的空白部分树的父指针表示法的数组表示。也就是所有等价对都被处理之后, 所得父结点的下标值(没有父结点则填“-1”)。

父结点下标

结点值

0 1

2 3

4 5

7 8

9 10 11 12 13 14 15

结点的下标 0 1

2 3

4 5

7 8

9 10 11 12 13 14 15

三、算法填空题（每题 12 分，共 24 分）

1.

（每空 3 分，共 12 分）下面的算法利用一个栈将一给定的序列从小到大排序。长度为 n 的序列由 $1, 2, \dots, n$ 这 n 个连续的正整数组成。请补全下面的代码段，使其可以判断给定的序列是否可以利用一个栈进行排序，如果可以，输出相应的操作。

例如：序列 4 3 1 2，此序列可以排序，输出：push push push pop push pop pop pop。

template <class T> // 栈的元素类型为 T

class Stack {

public:

def clear();

// 变为空栈

def push(const T item);

// item 入栈

T pop();

// 返回栈顶内容并弹出

T top();

// 返回栈顶内容但不弹出

isEmpty();

// 若栈已空返回真

isFull();

// 若栈已满返回真

};

Stack<int> s;

// s 为栈

int sequence[maxLen], len; // sequence 为待排序序列，len 为序列长度

islegal() {

// 判断序列是否可以排序

int i, j, k;

for (i = 0; i < len; i++) {

for (j = i+1; j < len; j++) {

for (k = j+1; k < len; k++) {

if ()

return False;

}

}

}

return True;

}

def transform() {

// 输出转换操作

int cur = 1, i = 0;

while () {

while (s.isEmpty() or) {

cout << "push ";

}

s.pop();

print("pop", end=" ")

++i;

}

}

2.

(每空 3 分, 分析 3 分, 共 12 分) 以下算法用来判断两棵树是否相同, 试补全下面的代码段, 并分析算法的运行时间代价。

```
template <class T>
IsEqual(TreeNode<T> *root1, TreeNode<T> *root2) {
while (root1 != None and root2 != None) {
if (root1.value() != root2.value())
return _____ ;
if (_____)
return False;
root1 = root1.RightSibling();
root2 = root2.RightSibling();
}
if (_____)
return True;
return False;
}
```

四、

分析证明题 (共 20 分)

1.

(5 分) 请证明非空满 K 叉树的叶结点数目为 $(K-1)*n+1$, 其中 n 为分支结点数目。

2.

(15 分) 有 n 个数 $K_1, K_2, \dots, K_n$ 顺序存储在数组中, 形成一棵 n 个结点的完全二叉树。现在要将它按如下方式建成一个最小值堆: 从左至右扫描整个数组, 将第 i 个元素 K_i ($i = 1, 2, \dots, n$) 与其父结点 $K[\lfloor i/2 \rfloor]$ 比较, 如果 $K_i \geq K[\lfloor i/2 \rfloor]$, 则不再进行操作; 若 $K_i < K[\lfloor i/2 \rfloor]$, 将 2 个结点对换, 再将 $K[\lfloor i/2 \rfloor]$ 与 $K[\lfloor i/4 \rfloor]$ 比较, 以此类推, 直到比较到根结点 K_1 。

(1) 按照这种建堆方式, 最坏情况下建堆的时间复杂度是多少?

(2) 教材上使用的“筛选法”建堆 (Sift down), 最坏情况下建堆的时间复杂度是多少?

(3) 上述 2 种建堆法有类似之处, 请问它们的时间复杂度是否一样, 若不同, 则请说明导致它们时间复杂度不同的原因。

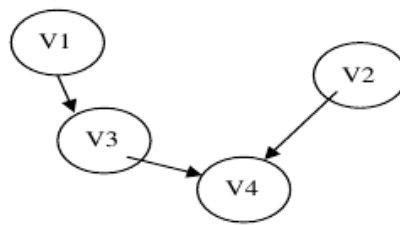
数据结构与算法 A

16 秋-期末

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择填空题（每空 1 分，共 11 分）

1. G 是一个非连通无向图，共有 21 条边，则图 G 至少有 8 个顶点。
2. 对于一个包含 N ($N > 1$) 个顶点的图，假定任意两点间最多只有一条边，那么下列哪些情况是错误的 AB。
A. 如果是有向图，则其任何一个极大强连通子图都无法进行拓扑排序。
B. 如果是无向连通图，则其最小生成树一定不包括权重最大的边。
C. 如果是无向连通图，假设所有边的权重均为正值，Dijkstra 算法给出的生成树不一定是最小生成树，但是与该图的任何一个最小生成树都至少有一条相同边。
3. 有向图 G 如下图所示：



- (1) 写出所有可能的拓扑序列：_____。
- (2) 若要使该图只有唯一的拓扑序列，则可以添加一条弧_____。
4. 在快速排序中，定义一次平分的划分为“幸运的划分”，而一次划分如果有一边为空则是“不幸的划分”。假设划分的过程总是“幸运”和“不幸”交替的，则该快速排序的时间复杂性为 $n \log n$ 。
5. 具有 10000 个关键码的 16 阶 B 树的查找路径长度（从根到叶节点访问 B 树索引块的次数）不会小于 3。
6. 设有 8 个初始归并段，其长度分别为 32, 46, 56, 64, 20, 87, 70, 40；进行 3 路归并排序，所构造的最佳归并树对应的总读写次数为 1590。
7. $A[N][N]$ 是对称矩阵，现将下三角矩阵按行存储到一维数组 $T[N(N+1)/2]$ 中（包括对角线），则对任一上三角元素 $A[i][j]$ 其对应值 ($0 \leq i \leq j < N$) 在 $T[k]$ 中的下标 k 是 $j(j+1)/2 + i$ 。
8. 在一棵空 AVL 树中，顺序插入如下关键码：{5, 9, 4, 2, 1, 3, 8}，请问全部插入后，在等概率下查找成功的平均检索长度为 $17/7$ 。
9. 已知广义表 $C = (c, (d, A), B, e)$ ，则广义表 C 的深度为 2， $\text{tail}(\text{head}(\text{tail}(C)))$ 的运算结果为 (A)。

二、简答辨析题（每题 3 分，共 15 分）

1. 如果要找出一个具有 n 个元素集合中的第 k ($1 \leq k \leq n$) 个最小元素，所学过的排序方法中哪种最适合？给出实现的基本思想。

三、算法填空题（每空 1 分，共 5 分）

1. 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下：

```
Class Graph {  
public:
```

```

int VerticesNum();
int EdgesNum();
Edge FirstEdge(int oneVertex);
Edge NextEdge(Edge preEdge);
bool IsEdge(Edge onEdge);
int FromVertex(Edge oneEdge);
int ToVertex(Edge oneEdge);
};
int visited[MAXSIZE]; //初始化为 0
int GetPathNum_Len(Graph& G, int i, int j, int len) {
if ( i == j && len == 0 ) return 1;
sum = 0; //sum 表示通过本结点的路径数
visited[i] = 1;
for (Edge e = G.FirstEdge(i); G.IsEdge(e); e = G.NextEdge(e)) {
int v = G.ToVertex(e);
if (!visited[v])
sum += GetPathNum_Len(G, v, j, len - 1) ;
} // for
visited[i] = 0; //本题允许曾经被访问过的结点出现在另一条路径中
return sum;
} // GetPathNum_Len

```

2. 下面的代码实现了一种计数排序：对每个待排序记录，扫描整个序列统计比它小的记录个数 count，count 即是该记录在序列中的争取位置。请将下面是计数排序的程序代码补充完整。

```

Template <class Record> void Sort(Record Array[], int n){
int curIndex = 0; //待排下标
while (curIndex < n) {
int count=0;
for ( int j=0;j<n;j++) //统计比 Array[curIndex] 小的值的个数
if (Array[j] < Array[curIndex])
count++;
swap(Array, curIndex, count);
if (curIndex == count)
curIndex++;
}
}

```

四、设计分析题（共 9 分）

1. （3 分）某小区有 N 座别墅需要供水。在第 i 座别墅里挖井需要 w[i] 的费用，在第 i 座和第 j 座别墅之间铺水管需要 c[i][j] 的费用。给每座别墅供水，要么挖井、要么跟其他有井的别墅铺设连通的水管路径。请设计算法，求解使每座别墅都得到供水的最小费用方案。

2. （6 分）现有一个工资系统，请实现满足如下操作需求的 Splay 树，并给出算法的伪代码：（Splay 树根为 root，其他变量可以自己定义）

1) void insert(int w); 新加一位工资为 w 的员工（假设 w 没有重复值）

2) void fire(int t); 解雇工资大于 t 的员工

相关定义和函数说明如下：

伸展树是一种自平衡的 BST，数据结构如下：

```

struct TreeNode {
int wage; //表示员工工资
TreeNode * father,* left,* right;
};

```

可以直接使用的函数：

void Splay(TreeNode* x, TreeNode* f); //将 x 旋为 f 的子结点 (f 在 x 的祖先路)

//把 x 旋到根结点即 Splay(x, NULL)

TreeNode* find(int x, TreeNode* S); //表示在以 S 为根的树中查找元素 x 的位置，若找不到，返回 x 应该插入位置的父亲结点

void Delete(TreeNode* x); // 删除以 x 结点为根的子树

void insert(int w){

if (!root) {

root = new TreeNode(w);

return;

}

int salary = w;

TreeNode * tmp = find(wage, root);

if (tmp->wage > salary) {

tmp->left = new TreeNode(salary);

Splay(tmp->left, NULL);

}

else {

tmp->right = new TreeNode(salary);

Splay(tmp->right, NULL);

}

}

}

void fire(int t){

if (!root) return;

salary = t;

TreeNode * tmp = find(salary, root);

if (tmp->wage == salary){

Splay(tmp, NULL);

Delete(tmp->right);

}

else {

if (tmp->wage > salary) {

tmp->left = new TreeNode(salary);

Splay(tmp->left, NULL);

}

else {

tmp->right = new TreeNode(salary);

Splay(tmp->right, NULL);

}

Delete(root->right);

if (root->left) {

Splay(root->left, NULL);

Delete(root->right);

} else Delete(root);

}

}

五、期中考试题（共 25 分，成绩由助教登记）

六、上机考试题（共 35 分，成绩由助教登记）

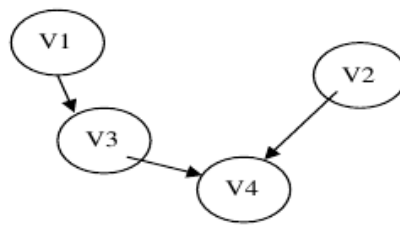
数据结构与算法 A

16 秋-期末

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择填空题（每空 1 分，共 11 分）

1. G 是一个非连通无向图，共有 21 条边，则图 G 至少有 8 个顶点。
2. 对于一个包含 N ($N > 1$) 个顶点的图，假定任意两点间最多只有一条边，那么下列哪些情况是错误的 AB。
A. 如果是有向图，则其任何一个极大强连通子图都无法进行拓扑排序。
B. 如果是无向连通图，则其最小生成树一定不包括权重最大的边。
C. 如果是无向连通图，假设所有边的权重均为正值，Dijkstra 算法给出的生成树不一定是最小生成树，但是与该图的任何一个最小生成树都至少有一条相同边。
3. 有向图 G 如下图所示：



- (1) 写出所有可能的拓扑序列：_____。
- (2) 若要使该图只有唯一的拓扑序列，则可以添加一条弧_____。
4. 在快速排序中，定义一次平分的划分为“幸运的划分”，而一次划分如果有一边为空则是“不幸的划分”。假设划分的过程总是“幸运”和“不幸”交替的，则该快速排序的时间复杂性为 $n \log n$ 。
5. 具有 10000 个关键码的 16 阶 B 树的查找路径长度（从根到叶节点访问 B 树索引块的次数）不会小于 3。
6. 设有 8 个初始归并段，其长度分别为 32, 46, 56, 64, 20, 87, 70, 40；进行 3 路归并排序，所构造的最佳归并树对应的总读写次数为 1590。
7. $A[N][N]$ 是对称矩阵，现将下三角矩阵按行存储到一维数组 $T[N(N+1)/2]$ 中（包括对角线），则对任一上三角元素 $A[i][j]$ 其对应值 ($0 \leq i \leq j < N$) 在 $T[k]$ 中的下标 k 是 $j(j+1)/2 + i$ 。
8. 在一棵空 AVL 树中，顺序插入如下关键码：{5, 9, 4, 2, 1, 3, 8}，请问全部插入后，在等概率下查找成功的平均检索长度为 $17/7$ 。
9. 已知广义表 $C = (c, (d, A), B, e)$ ，则广义表 C 的深度为 2， $\text{tail}(\text{head}(\text{tail}(C)))$ 的运算结果为 (A)。

二、简答辨析题（每题 3 分，共 15 分）

1. 如果要找出一个具有 n 个元素集合中的第 k ($1 \leq k \leq n$) 个最小元素，所学过的排序方法中哪种最适合？给出实现的基本思想。

三、算法填空题（每空 1 分，共 5 分）

1. 下述算法实现在图 G 中计算从顶点 i 到顶点 j 之间长度为 len 的简单路径条数。图的 ADT 如下：

```
Class Graph {  
public:
```

```

int VerticesNum();
int EdgesNum();
Edge FirstEdge(int oneVertex);
Edge NextEdge(Edge preEdge);
IsEdge(Edge onEdge);
int FromVertex(Edge oneEdge);
int ToVertex(Edge oneEdge);
};
int visited[MAXSIZE]; //初始化为 0
int GetPathNum_Len(G, int i, int j, int len) {
if ( i == j and len == 0 ) return 1;
sum = 0; //sum 表示通过本结点的路径数
visited[i] = 1;
for (Edge e = G.FirstEdge(i); G.IsEdge(e); e = G.NextEdge(e)) {
int v = G.ToVertex(e);
if (not visited[v])
sum += GetPathNum_Len(G, v, j, len - 1) ;
} // for
visited[i] = 0; //本题允许曾经被访问过的结点出现在另一条路径中
return sum;
} // GetPathNum_Len

```

2. 下面的代码实现了一种计数排序：对每个待排序记录，扫描整个序列统计比它小的记录个数 count，count 即是该记录在序列中的争取位置。请将下面是计数排序的程序代码补充完整。

```

Template <class Record> def Sort(Record Array[], int n){
int curIndex = 0; //待排下标
while (curIndex < n) {
int count=0;
for ( int j=0;j<n;j++) //统计比 Array[curIndex] 小的值的个数
if (Array[j] < Array[curIndex])
count++;
swap(Array, curIndex, count);
if (curIndex == count)
curIndex++;
}
}

```

四、设计分析题（共 9 分）

1. （3 分）某小区有 N 座别墅需要供水。在第 i 座别墅里挖井需要 w[i] 的费用，在第 i 座和第 j 座别墅之间铺水管需要 c[i][j] 的费用。给每座别墅供水，要么挖井、要么跟其他有井的别墅铺设连通的水管路径。请设计算法，求解使每座别墅都得到供水的最小费用方案。

2. （6 分）现有一个工资系统，请实现满足如下操作需求的 Splay 树，并给出算法的伪代码：（Splay 树根为 root，其他变量可以自己定义）

1) def insert(int w); 新加一位工资为 w 的员工（假设 w 没有重复值）

2) def fire(int t); 解雇工资大于 t 的员工

相关定义和函数说明如下：

伸展树是一种自平衡的 BST，数据结构如下：

```

class TreeNode {
int wage; //表示员工工资
TreeNode * father,* left,* right;
};

```

可以直接使用的函数：

```
def Splay(TreeNode* x, TreeNode* f); //将 x 旋为 f 的子结点 (f 在 x 的祖先路)
```

```
//把 x 旋到根结点即 Splay(x, None)
```

```
TreeNode* find(int x, TreeNode* S); //表示在以 S 为根的树中查找元素 x  
的位置，若找不到，返回 x 应该插入位置的父亲结点
```

```
def Delete(TreeNode* x); // 删除以 x 结点为根的子树
```

```
def insert(int w){
```

```
if (root is None) {
```

```
root = TreeNode(w);
```

```
return;
```

```
}
```

```
int salary = w;
```

```
TreeNode * tmp = find(wage, root);
```

```
if (tmp.wage > salary) {
```

```
tmp.left = TreeNode(salary);
```

```
Splay(tmp.left, None);
```

```
}
```

```
else {
```

```
tmp.right = TreeNode(salary);
```

```
Splay(tmp.right, None);
```

```
}
```

```
}
```

```
}
```

```
def fire(int t){
```

```
if (root is None) return;
```

```
salary = t;
```

```
TreeNode * tmp = find(salary, root);
```

```
if (tmp.wage == salary){
```

```
Splay(tmp, None);
```

```
Delete(tmp.right);
```

```
}
```

```
else {
```

```
if (tmp.wage > salary) {
```

```
tmp.left = TreeNode(salary);
```

```
Splay(tmp.left, None);
```

```
}
```

```
else {
```

```
tmp.right = TreeNode(salary);
```

```
Splay(tmp.right, None);
```

```
}
```

```
Delete(root.right);
```

```
if (root.left) {
```

```
Splay(root.left, None);
```

```
Delete(root.right);
```

```
} else Delete(root);
```

```
}
```

```
}
```

五、期中考试题（共 25 分，成绩由助教登记）

六、上机考试题（共 35 分，成绩由助教登记）

数据结构与算法 A

16 秋-期中

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择题（每题 2 分，共 20 分）

1. 下面函数的时间复杂度是_____。

```
int work(int n, int m) {  
    int sum = 0;  
    for (int i = 1; i <= m; i *= 2)  
        for (int j = 1; j <= n; j *= 2) {  
            sum += i * j;  
        }  
    return sum;  
}
```

2. 下面关于线性表的叙述中，正确的是哪些？

- A. 采用顺序存储的线性表，必须占用一片连续的存储单元。
- B. 采用顺序存储的线性表，便于进行插入和删除操作。
- C. 采用链接存储的线性表，不必占用一片连续的存储单元。
- D. 采用链接存储的线性表，便于插入和删除操作。
- E. 采用链接存储的线性表，插入第 i 个元素的时间与 i 的数值成正比。
- F. 采用链接存储的线性表，在已知的结点 p 后插入一个新节点的时间复杂度与结点 p 的位置有关。

3. 若元素 a 、 b 、 c 、 d 依次进栈，则可能的出栈序列有_____种；若有元素 a 、 b 、 c 、 d 、 e 依次进栈，则可能的出栈序列有_____种。

4. 一个字符串中_____称为该串的子串。 $abcbab$ 的不相等真子串个数为_____。

5. 一个具有 n 个结点的二叉树的叶结点的个数最多为_____。

6. 一个有 5 层结点的完全 3 叉树。按前序遍历周游给结点从 1 开始编号，则第 100 号结点的父结点的编码为_____。（注释：独根树为 1 层的）

7. 将序列 $\{12, 70, 33, 65, 24, 56, 48, 92, 86, 33\}$ 按建堆方法调整为小值堆，调整后的序列为_____。

8. 假设用于通信的电文仅由 8 个字符 $\{a, b, c, d, e, f, g, h\}$ 组成，字符在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10，对这些字符进行哈夫曼编码的外部路径长度为_____。

9. 将森林 F 转换为对应的二叉树 T ， F 中的叶结点的个数为_____。

- A. T 中叶结点的个数
- B. T 中度为 1 的结点个数
- C. T 中左子指针为空的结点个数
- D. T 中右子指针为空的结点个数

10. 一棵树 T 有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的结点个数为_____。

二、辨析与简答 (共 32 分)

1. (6 分) 请谈谈循环队列和链表的优缺点。当遇到如下情况时，应当使用哪一种数据结构？

情况：你要维护食堂鸡腿饭的排队队伍，队伍有两种操作：1. 排在队首的人出来打鸡腿饭后离开；2. 一个新来的人排在队尾。你知道食堂不太大，所以队伍的最大长度你可以估计出来。

2. (4 分) 如何找出具有相同的中序序列和后序序列的所有二叉树？

3. (4 分) 从空二叉树开始，将关键码 $\{18, 73, 10, 5, 68, 99, 27, 41, 51, 32, 25\}$ 严格按

照 BST（二叉搜索树）的插入算法逐个插入 BST，画出这样构造出的 BST 树，并写出对该 BST 按照前序遍历得到的序列。

4.

（6 分）以下是计算模式 P 的 next 向量的算法：

```
int *findNext(string P) {
    int i = 0, k = -1;
    int m = P.length();
    int* next = new int[m];
    next[0] = -1;
    while (i < m) {
        while (k >= 0 && P[i] != P[k])
            k = next[k];
        i++, k++;
        if (i == m) break;
        if (P[i] == P[k])
            next[i] = next[k];
        else
            next[i] = k;
    }
    return next;
}
```

采用上述算法，计算模式 P="abcdaabcb" 的 next 向量。

i

P[i]

a

b

c

d

a

a

b

c

a

b

next[i]

5.

（6 分）数组 {23, 17, 14, 6, 13, 10, 1, 5, 8, 12} 是否为一个最大值堆？若是，请说明理由，否则请严格按照筛选法建堆的过程将其调整成为最大值堆，并画出逐步调整建堆的过程。

6.

（6 分）对下列 15 个等价对进行合并，给出所得等价类树的图示。在初始情况下，集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则，合并时子树结点少的并入结点数多的那棵（多的那个作为新树根，少的那个根作为新根的直接子结点）；若两棵树规模同样大，则把根值较大的并入根值较小（新树根取值小的）。同时，采用路径压缩优化。
(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9) (4,14) (6,7) (8,10)
(8,7) (7,0) (10,15) (10,13)

三、算法填空（每空 2 分，共 16 分）

1. 下面的算法将一个用带度数的后根次序法表示的森林转换为左子结点/右兄弟结点法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。空格中可能需要填写 0 到多条语句（或表达式）。

```

template<class value_type>
class stack {
public:
bool empty(); // 判断栈空
int size(); // 返回栈大小
value_type &top(); // 读栈顶
void push(const value_type& X); // 入栈
void pop(); // 出栈
};

struct Node<T> {
T info; // 结点的数据信息
int degree; // 结点的度数信息
};

template<class T>
class TreeNode { // 树结点类
public:
bool isLeaf(); // 判断当前结点是否为叶结点
T Value(); // 返回结点的值
TreeNode<T> *LeftMostChild(); // 返回第一个子结点
TreeNode<T> *RightSibling(); // 返回右兄
void setValue(const T&); // 设置当前结点的值
void setChild(TreeNode<T> *pointer); // 设置左子
void setSibling(TreeNode<T> *pointer); // 设置右兄
};

template<class T>
TreeNode<T> *Convert(Node<T>* nodes, int size) {
TreeNode<T> *cur, *temp1, *temp2;
stack<TreeNode<T>*> Cstack;
for ( int i = 0; i < size; i++) {
cur = new TreeNode<T>(nodes[i].info);
if ( nodes[i].degree == 0 )
// 填空 1
else {
assert( nodes[i].degree <= Cstack.size() );
temp2 = NULL;
for (int j = 0; // 填空 2 ; j++) {
temp1 = Cstack.top();
Cstack.pop();
// 填空 3
temp2 = temp1;
}
// 填空 4
Cstack.push(cur);
}
}
cur = temp2 = NULL;
while( !Cstack.empty() ) {
cur = Cstack.top();
Cstack.pop();
// 填空 5
temp2 = cur;
}

```

```
}
return cur;
}
2. 给定一个二叉树的根结点和树中任意两个结点，补全下面的程序段以求这两个结点的最近公共祖先。
TreeNode* lowestCommonAncestor(
TreeNode* root, TreeNode* p, TreeNode* q) {
if ( // 填空 6 ) return root;
TreeNode* left = lowestCommonAncestor(root->left, p, q);
TreeNode* right = lowestCommonAncestor(root->right, p, q);
if ( __ // 填空 7 __ ) return root;
return __ // 填空 8 __ ? left:right;
}
```

四、算法设计与实现 (16 分)

1.

(8 分) 现有两个满足先进先出原则的队列 A 和 B，在队列上可进行以下 4 个操作：

front(): 获得队首元素

push_back(T x): 把元素 x 插到队尾。

pop_front(): 删除队首元素。

empty(): 判断是否为空。

请用 A 和 B 模拟一个栈的 4 个操作：top(), pop(), push(), empty()。只要给出实现栈操作的基本思想，对时间效率无具体要求。

2.

(8 分) 对于两个串 A 和 B，给出一种算法使得能够保证正确地求出最大的 L 值，使得 A 串中长为 L 的前缀与 B 串中长为 L 的后缀相等。

五、分析证明题 (共 16 分)

1.

(6 分) 试证明在一棵树中，u 是 v 的祖先，当且仅当在先序遍历中 u 在 v 之前且在后序遍历中 u 在 v 之后。

2.

(10 分) 对于满二叉树：

(1) 试问具有 n 个叶结点的树中共有多少个结点？

(2) 试证明：

，其中 n 为叶结点的个数，

il 表示第 i 个叶结点所在的

层次（设根结点所在的层次为 0）。

数据结构与算法 A

16 秋-期中

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择题（每题 2 分，共 20 分）

1. 下面函数的时间复杂度是_____。

```
int work(int n, int m) {  
    int sum = 0;  
    for (int i = 1; i <= m; i *= 2)  
        for (int j = 1; j <= n; j *= 2) {  
            sum += i * j;  
        }  
    return sum;  
}
```

2. 下面关于线性表的叙述中，正确的是哪些？

- A. 采用顺序存储的线性表，必须占用一片连续的存储单元。
- B. 采用顺序存储的线性表，便于进行插入和删除操作。
- C. 采用链接存储的线性表，不必占用一片连续的存储单元。
- D. 采用链接存储的线性表，便于插入和删除操作。
- E. 采用链接存储的线性表，插入第 i 个元素的时间与 i 的数值成正比。
- F. 采用链接存储的线性表，在已知的结点 p 后插入一个新节点的时间复杂度与结点 p 的位置有关。

3. 若元素 a 、 b 、 c 、 d 依次进栈，则可能的出栈序列有_____种；若有元素 a 、 b 、 c 、 d 、 e 依次进栈，则可能的出栈序列有_____种。

4. 一个字符串中_____称为该串的子串。 $abcbab$ 的不相等真子串个数为_____。

5. 一个具有 n 个结点的二叉树的叶结点的个数最多为_____。

6. 一个有 5 层结点的完全 3 叉树。按前序遍历周游给结点从 1 开始编号，则第 100 号结点的父结点的编码为_____。（注释：独根树为 1 层的）

7. 将序列 $\{12, 70, 33, 65, 24, 56, 48, 92, 86, 33\}$ 按建堆方法调整为小值堆，调整后的序列为_____。

8. 假设用于通信的电文仅由 8 个字符 $\{a, b, c, d, e, f, g, h\}$ 组成，字符在电文中出现的频率分别为 0.07, 0.19, 0.02, 0.06, 0.32, 0.03, 0.21, 0.10，对这些字符进行哈夫曼编码的外部路径长度为_____。

9. 将森林 F 转换为对应的二叉树 T ， F 中的叶结点的个数为_____。

- A. T 中叶结点的个数
- B. T 中度为 1 的结点个数
- C. T 中左子指针为空的结点个数
- D. T 中右子指针为空的结点个数

10. 一棵树 T 有 20 个度为 4 的结点，10 个度为 3 的结点，1 个度为 2 的结点，10 个度为 1 的结点，则树 T 的结点个数为_____。

二、辨析与简答 (共 32 分)

1. (6 分) 请谈谈循环队列和链表的优缺点。当遇到如下情况时，应当使用哪一种数据结构？

情况：你要维护食堂鸡腿饭的排队队伍，队伍有两种操作：1. 排在队首的人出来打鸡腿饭后离开；2. 一个新来的人排在队尾。你知道食堂不太大，所以队伍的最大长度你可以估计出来。

2. (4 分) 如何找出具有相同的中序序列和后序序列的所有二叉树？

3. (4 分) 从空二叉树开始，将关键码 $\{18, 73, 10, 5, 68, 99, 27, 41, 51, 32, 25\}$ 严格按

照 BST（二叉搜索树）的插入算法逐个插入 BST，画出这样构造出的 BST 树，并写出对该 BST 按照前序遍历得到的序列。

4.

（6 分）以下是计算模式 P 的 next 向量的算法：

```
int *findNext(P) {
    int i = 0, k = -1;
    int m = len(P);
    int* next = int[m];
    next[0] = -1
    while i < m:
        while (k >= 0 and P[i] != P[k])
            k = next[k];
        i++, k++;
        if i == m: break
        if (P[i] == P[k])
            next[i] = next[k];
        else
            next[i] = k;
    }
    return next;
}
```

采用上述算法，计算模式 P="abcdaabcb" 的 next 向量。

i

P[i]

a

b

c

d

a

a

b

c

a

b

next[i]

5.

（6 分）数组 {23, 17, 14, 6, 13, 10, 1, 5, 8, 12} 是否为一个最大值堆？若是，请说明理由，否则请严格按照筛选法建堆的过程将其调整成为最大值堆，并画出逐步调整建堆的过程。

6.

（6 分）对下列 15 个等价对进行合并，给出所得等价类树的图示。在初始情况下，集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则，合并时子树结点少的并入结点数多的那棵（多的那个作为新树根，少的那个根作为新根的直接子结点）；若两棵树规模同样大，则把根值较大的并入根值较小（新树根取值小的）。同时，采用路径压缩优化。
(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (3,9) (4,14) (6,7) (8,10)
(8,7) (7,0) (10,15) (10,13)

三、算法填空（每空 2 分，共 16 分）

1. 下面的算法将一个用带度数的后根次序法表示的森林转换为左子结点/右兄弟结点法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。空格中可能需要填写 0 到多条语句（或表达式）。

```

template<class value_type>
class stack {
public:
empty(); // 判断栈空
int size(); // 返回栈大小
value_type &top(); // 读栈顶
def push(const value_type& X); // 入栈
def pop(); // 出栈
};
class Node<T> {
T info; // 结点的数据信息
int degree; // 结点的度数信息
};
template<class T>
class TreeNode { // 树结点类
public:
isLeaf(); // 判断当前结点是否为叶结点
T Value(); // 返回结点的值
TreeNode<T> *LeftMostChild(); // 返回第一个子结点
TreeNode<T> *RightSibling(); // 返回右兄
def setValue(const T&); // 设置当前结点的值
def setChild(TreeNode<T> *pointer); // 设置左子
def setSibling(TreeNode<T> *pointer); // 设置右兄
};
template<class T>
TreeNode<T> *Convert(Node<T>* nodes, int size) {
TreeNode<T> *cur, *temp1, *temp2;
stack<TreeNode<T>*> Cstack;
for ( int i = 0; i < size; i++) {
cur = TreeNode<T>(nodes[i].info);
if ( nodes[i].degree == 0 )
// 填空 1
else {
assert( nodes[i].degree <= Cstack.size() );
temp2 = None;
for (int j = 0; // 填空 2 ; j++) {
temp1 = Cstack.top();
Cstack.pop();
// 填空 3
temp2 = temp1;
}
// 填空 4
Cstack.push(cur);
}
}
cur = temp2 = None;
while( not Cstack.empty() ) {
cur = Cstack.top();
Cstack.pop();
// 填空 5
temp2 = cur;
}

```

```
}  
return cur;  
}  
2. 给定一个二叉树的根结点和树中任意两个结点，补全下面的程序段以求这两个结点的最近公共祖先。  
TreeNode* lowestCommonAncestor(  
    TreeNode* root, TreeNode* p, TreeNode* q) {  
    if ( // 填空 6 ) return root;  
    TreeNode* left = lowestCommonAncestor(root.left, p, q);  
    TreeNode* right = lowestCommonAncestor(root.right, p, q);  
    if ( __ // 填空 7 __ ) return root;  
    return __ // 填空 8 __ ? left:right;  
}
```

四、算法设计与实现 (16 分)

1.

(8 分) 现有两个满足先进先出原则的队列 A 和 B，在队列上可进行以下 4 个操作：

front(): 获得队首元素

push_back(T x): 把元素 x 插到队尾。

pop_front(): 删除队首元素。

empty(): 判断是否为空。

请用 A 和 B 模拟一个栈的 4 个操作：top(), pop(), push(), empty()。只要给出实现栈操作的基本思想，对时间效率无具体要求。

2.

(8 分) 对于两个串 A 和 B，给出一种算法使得能够保证正确地求出最大的 L 值，使得 A 串中长为 L 的前缀与 B 串中长为 L 的后缀相等。

五、分析证明题 (共 16 分)

1.

(6 分) 试证明在一棵树中，u 是 v 的祖先，当且仅当在先序遍历中 u 在 v 之前且在后序遍历中 u 在 v 之后。

2.

(10 分) 对于满二叉树：

(1) 试问具有 n 个叶结点的树中共有多少个结点？

(2) 试证明：

，其中 n 为叶结点的个数，

il 表示第 i 个叶结点所在的

层次（设根结点所在的层次为 0）。

数据结构与算法 A

17 秋-期末

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择填空题（7 小题，每空 1 分，共 10 分）

1.

下列说法中正确的是_____。

- A. 图的广度优先搜索中邻接点的寻找具有“先进后出”的特征。
- B. 连通图的生成树是该图的一个极大连通子图。
- C. 图的广度优先搜索是一个递归过程。
- D. 在非连通图的遍历过程中，每调用一次深度优先搜索算法都得到该图的一个连通分量。

2.

基于比较的内排序算法中，平均时间复杂度为 $O(n \log n)$ 的算法有_____，这其中稳定的排序算法有_____。

3.

在包含 n 个关键码的线性表中从后向前进行顺序检索（0 下标为监视哨）。若第 i 个关键码的检索概率为 p_i ，且满足如下分布：

$$p_1 = 1/2, p_2 = 1/4, \dots, p_n = 1/2$$

 n ,

则成功检索的平均检索长度为_____，失败检索的平均检索长度为_____。

4.

给定一组输入关键码 {15, 1, 3, 14, 11, 2, 12, 10, 4, 17}，采用大小为 3 的最小堆进行置换选择排序的输出为_____。

5.

按照 AVL 定义，下图所示的 BST 中没有达到平衡状态的结点有_____。

3 / 6

6.

假设磁盘块大小是 1024 字节，每个索引项需要 8 个字节。一个包含 20000 条记录的数据文件，若采用二级索引，则一级索引文件和二级索引文件共需占用_____磁盘块。

7.

给定两个广义表 $S = ((a, (b), c), ((d), e))$, $T = (f, (g, ((h)), i, j))$ ；利用广义表的 Head 和 Tail 运算，则有

$d =$ _____；

$h =$ _____。

二、简答辨析题（5 小题，共 15 分）

1. （2 分）将关键码序列（7, 8, 30, 11, 18, 9, 14）存储到一个散列表中。

散列表是一个下标从 0 开始的一维数组，若散列函数采用： $H(\text{key}) = (\text{key} * 3) \bmod 7$ ，处理冲突采用线性探测法，装填（负载）因子要求为 0.7。

- (1) . 请画出所构造的散列表；
- (2) . 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

2. （4 分）下图是一棵关键码为英文字符的 m 阶 B 树，依据字典序组织。请回答下述问题：

- (1) . 这是一棵合法的 B 树，则 m 可以取哪些值，并说明原因；

(2) . 若 m 取所有可能值中的最小值, 则

(A) 给出查找字符 V 的过程, 并说明需要的读盘次数 (假设根结点也需读盘操作);

(B) 给出插入字符 I ($H < I < J$) 的过程, 并画出插入 I 后的 B 树;

(C) 在原始 B 树 (即进行第 (B) 问的操作之前) 的基础上, 给出删除 H 的过程, 并画出删除 H 后的 B 树。

3. (4 分) 现有 8 个顺串, 每个顺串的第二个关键码为 $S1[1..8] = \{17, 10, 2, 9, 13, 15, 12, 4\}$, 第二个关键码是 $S2[1..8] = \{23, 34, 38, 25, 27, 40, 36, 39\}$, 进行从小到大的归并。

(1) . 请写出初始构建的赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点 4 / 6

的数组 $B[0..7]$ (数组 A 和 B 存储的是 1 到 8 的顺串的下标, $B[0]$ 表示最终冠军);

(2) . 在输出第一个全局赢者并重构赢者树或败者树后, 写出此时赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点的数组 $B[0..7]$ 。

4. (3 分) 请按照 2, 1, 4, 5, 9 的插入顺序, 建立一棵红黑树, 并画出每插入一个元素后的红黑树。(用 B 和 R 表示结点的颜色)

5. (2 分) 给定下面的一棵 splay 树, 按照结点序列 2, 3, 4, 5, 6, 7 访问所有结点之后, 请画出访问后的 splay 树形结构。

三、算法填空题 (2 小题, 每空 1 分, 共 5 分)

1.

下面是考虑墓碑问题的散列表插入算法, 算法中不允许有重复关键码插入。

```
template <class Key, class Elem, class KEComp, class EECComp> class
```

```
hashdict {
```

```
private:
```

```
Elem* HT;
```

```
// 散列表
```

```
int M;
```

```
// 散列表大小
```

```
int current;
```

```
// 现有元素数目
```

```
Elem EMPTY;
```

```
// 空槽
```

```
int p(Key K, int i)
```

```
// 探查函数
```

```
int h(Key K) const ;
```

```
// 散列函数
```

```
Key getKey(Elem e);
```

```
// 获取元素 e 的关键码值
```

```
public:
```

```
hashdict(int sz, Elem e){
```

```
// 构造函数, e 用来定义空槽
```

```
M = sz;
```

```
EMPTY = e;
```

```
current = 0;
```

```
HT = new Elem[sz]
```

```
for (int i=0; i<M; i++){
```

```
HT[i] = EMPTY;
```

```
}
```

```
}
```

```
~hashdict() { delete []HT; }
```

5 / 6

```
bool hashSearch(const Key&, Elem&) const;
bool hashInsert(const Elem&);
Elem hashDelete(const Key& K);
int size() { return current; }
// 散列表中现有元素数
};
// 墓碑用常量 TOMB 表示，以下为插入函数
template<class Key,class Elem,class KEComp,class EEComp>
bool hashdict<Key,Elem,KEComp,EEComp>::hashInsert(const Elem& e){
int insplace;
int i = 0;
int pos = home = h(getkey(e)); // 找到新结点基地址
bool tomb_pos = false;
while (_____ 填空 1 _____) {
if (EEComp::eq(e,HT[pos]))
return false;
if (EEComp::eq(TOMB,HT[pos]) && !tomb_pos) {
_____ 填空 2 _____
tomb_pos = true;
}
i++;
_____ 填空 3 _____ // 探查
}
if (!tomb_pos)
insplace = pos;
// 若无墓碑，则 insplace 位于空槽位置
HT[insplace] = e;
return true;
}
```

2.

设有一个仅由红、白、蓝 3 种颜色的条块组成的序列，各种色块的个数和分布是随机的，但 3 种颜色的总数为 n 。下面的代码实现了一种时间复杂度为 $O(n)$ 的排序，使得这些条块按照红、白、蓝的顺序排好。请将其补全。

```
const int Red = 1, White = 2, Blue = 3;
void ColorSort(int Array[], int n) {
int Insert_Red = 0, Insert_White = 0, i;
for (i=0; i<n; i++) {
if (Array[i] == White) {
Array[i] = Array[Insert_White];
Array[Insert_White] = White;
Insert_White++;
}
if (Array[i] == Red) {
_____ 填空 4 _____
_____ 填空 5 _____
Array[Insert_Red] = Red;
```

6 / 6

```
Insert_Red++;
Insert_White++;
}
```

```
}  
}
```

四、设计分析题（2 小题，共 10 分）

1.

（6 分）请设计一个算法求有向无环图（DAG）中每对顶点间的“最长简单路径”（所谓“最长简单路径”，是指该简单路径包含的边数最多）。以一个有向无环图作为输入，对于每对顶点，如果它们之间存在简单路径，则输出其中路径长度最长（边数最多）的简单路径，否则输出空。试分析你所设计算法的时间复杂度。

2.

（4 分）利用证明基于比较的排序问题的复杂度下限的方法，试证明在已排序序列上基于比较的检索问题的复杂度下限是 $\Omega(\log n)$ 。

五、期中考试题（共 30 分，成绩由助教登记）

六、上机考试题（共 30 分，成绩由助教登记）

数据结构与算法 A

17 秋-期末

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择填空题（7 小题，每空 1 分，共 10 分）

1.

下列说法中正确的是_____。

- A. 图的广度优先搜索中邻接点的寻找具有“先进后出”的特征。
- B. 连通图的生成树是该图的一个极大连通子图。
- C. 图的广度优先搜索是一个递归过程。
- D. 在非连通图的遍历过程中，每调用一次深度优先搜索算法都得到该图的一个连通分量。

2.

基于比较的内排序算法中，平均时间复杂度为 $O(n \log n)$ 的算法有_____，这其中稳定的排序算法有_____。

3.

在包含 n 个关键码的线性表中从后向前进行顺序检索（0 下标为监视哨）。若第 i 个关键码的检索概率为 p_i ，且满足如下分布：

$$p_1 = 1/2, p_2 = 1/4, \dots, p_n = 1/2^n,$$

 n ,

则成功检索的平均检索长度为_____，失败检索的平均检索长度为_____。

4.

给定一组输入关键码 {15, 1, 3, 14, 11, 2, 12, 10, 4, 17}，采用大小为 3 的最小堆进行置换选择排序的输出为_____。

5.

按照 AVL 定义，下图所示的 BST 中没有达到平衡状态的结点有_____。

3 / 6

6.

假设磁盘块大小是 1024 字节，每个索引项需要 8 个字节。一个包含 20000 条记录的数据文件，若采用二级索引，则一级索引文件和二级索引文件共需占用_____磁盘块。

7.

给定两个广义表 $S = ((a, (b), c), ((d), e))$, $T = (f, (g, ((h)), i, j))$ ；利用广义表的 Head 和 Tail 运算，则有

$d =$ _____；

$h =$ _____。

二、简答辨析题（5 小题，共 15 分）

1. （2 分）将关键码序列（7, 8, 30, 11, 18, 9, 14）存储到一个散列表中。

散列表是一个下标从 0 开始的一维数组，若散列函数采用： $H(\text{key}) = (\text{key} * 3) \bmod 7$ ，处理冲突采用线性探测法，装填（负载）因子要求为 0.7。

- (1). 请画出所构造的散列表；
- (2). 分别计算等概率情况下查找成功和查找不成功的平均查找长度。

2. （4 分）下图是一棵关键码为英文字符的 m 阶 B 树，依据字典序组织。请回答下述问题：

- (1). 这是一棵合法的 B 树，则 m 可以取哪些值，并说明原因；

(2). 若 m 取所有可能值中的最小值, 则

(A) 给出查找字符 V 的过程, 并说明需要的读盘次数 (假设根结点也需读盘操作);

(B) 给出插入字符 I ($H < I < J$) 的过程, 并画出插入 I 后的 B 树;

(C) 在原始 B 树 (即进行第 (B) 问的操作之前) 的基础上, 给出删除 H 的过程, 并画出删除 H 后的 B 树。

3. (4 分) 现有 8 个顺串, 每个顺串的第一个关键码为 $S1[1..8] = \{17, 10, 2, 9, 13, 15, 12, 4\}$, 第二个关键码是 $S2[1..8] = \{23, 34, 38, 25, 27, 40, 36, 39\}$, 进行从小到大的归并。

(1). 请写出初始构建的赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点 4 / 6

的数组 $B[0..7]$ (数组 A 和 B 存储的是 1 到 8 的顺串的下标, $B[0]$ 表示最终冠军);

(2). 在输出第一个全局赢者并重构赢者树或败者树后, 写出此时赢者树内部结点的数组 $A[1..7]$ 和败者树内部结点的数组 $B[0..7]$ 。

4. (3 分) 请按照 2, 1, 4, 5, 9 的插入顺序, 建立一棵红黑树, 并画出每插入一个元素后的红黑树。(用 B 和 R 表示结点的颜色)

5. (2 分) 给定下面的一棵 splay 树, 按照结点序列 2, 3, 4, 5, 6, 7 访问所有结点之后, 请画出访问后的 splay 树形结构。

三、算法填空题 (2 小题, 每空 1 分, 共 5 分)

1.

下面是考虑墓碑问题的散列表插入算法, 算法中不允许有重复关键码插入。

```
template <class Key, class Elem, class KEComp, class EECComp> class
```

```
hashdict {
```

```
private:
```

```
Elem* HT;
```

```
// 散列表
```

```
int M;
```

```
// 散列表大小
```

```
int current;
```

```
// 现有元素数目
```

```
Elem EMPTY;
```

```
// 空槽
```

```
int p(Key K, int i)
```

```
// 探查函数
```

```
int h(Key K) const ;
```

```
// 散列函数
```

```
Key getKey(Elem e);
```

```
// 获取元素 e 的关键码值
```

```
public:
```

```
hashdict(int sz, Elem e){
```

```
// 构造函数, e 用来定义空槽
```

```
M = sz;
```

```
EMPTY = e;
```

```
current = 0;
```

```
HT = Elem[sz]
```

```
for (int i=0; i<M; i++){
```

```
HT[i] = EMPTY;
```

```
}
```

```
}
```

```
~hashdict() { delete []HT; }
```

5 / 6

```

hashSearch(const Key&, Elem&) const;
hashInsert(const Elem&);
Elem hashDelete(const Key& K);
int size() { return current; }
// 散列表中现有元素数
};
// 墓碑用常量 TOMB 表示，以下为插入函数
template<class Key,class Elem,class KEComp,class EEComp>
hashdict<Key,Elem,KEComp,EEComp>::hashInsert(const Elem& e){
int insplace;
int i = 0;
int pos = home = h(getkey(e)); // 找到新结点基地址
tomb_pos = false;
while (_____ 填空 1 _____) {
if (EEComp::eq(e,HT[pos]))
return False;
if (EEComp::eq(TOMB,HT[pos]) and !tomb_pos) {
_____ 填空 2 _____
tomb_pos = true;
}
i++;
_____ 填空 3 _____ // 探查
}
if (!tomb_pos)
insplace = pos;
// 若无墓碑，则 insplace 位于空槽位置
HT[insplace] = e;
return True;
}

```

2.

设有一个仅由红、白、蓝 3 种颜色的条块组成的序列，各种色块的个数和分布是随机的，但 3 种颜色的总数为 n 。下面的代码实现了一种时间复杂度为 $O(n)$ 的排序，使得这些条块按照红、白、蓝的顺序排好。请将其补全。

```

const int Red = 1, White = 2, Blue = 3;
def ColorSort(int Array[], int n) {
int Insert_Red = 0, Insert_White = 0, i;
for (i=0; i<n; i++) {
if (Array[i] == White) {
Array[i] = Array[Insert_White];
Array[Insert_White] = White;
Insert_White++;
}
if (Array[i] == Red) {
_____ 填空 4 _____
_____ 填空 5 _____
Array[Insert_Red] = Red;
}
}
Insert_Red++;
Insert_White++;
}

```

6 / 6

```
}  
}
```

四、设计分析题（2 小题，共 10 分）

1.

（6 分）请设计一个算法求有向无环图（DAG）中每对顶点间的“最长简单路径”（所谓“最长简单路径”，是指该简单路径包含的边数最多）。以一个有向无环图作为输入，对于每对顶点，如果它们之间存在简单路径，则输出其中路径长度最长（边数最多）的简单路径，否则输出空。试分析你所设计算法的时间复杂度。

2.

（4 分）利用证明基于比较的排序问题的复杂度下限的方法，试证明在已排序序列上基于比较的检索问题的复杂度下限是 $\Omega(\log n)$ 。

五、期中考试题（共 30 分，成绩由助教登记）

六、上机考试题（共 30 分，成绩由助教登记）

数据结构与算法 A

17 秋-期中

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择与填空（每空 2 分，共 24 分）

1.

下面函数的时间复杂度是 O

```
void recursive(int n, int m, int k){  
    if (n <= 0)  
        printf("%d, %d\n", m, k);  
    else {  
        recursive(n-1, m+1, k);  
        recursive(n-1, m, k+1);  
    }  
}
```

A. $O(n*m*k)$ B. $O(n^2*m^2)$

B.

 $O(2^n)$ D. $O(n!)$

2.

完成在双循环链表结点 p 之后插入 s 的操作为 ():

A. $p \rightarrow next \rightarrow prev = s$; $s \rightarrow prev = p$; $s \rightarrow next = p \rightarrow next$; $p \rightarrow next = s$;B. $p \rightarrow next \rightarrow prev = s$; $p \rightarrow next = s$; $s \rightarrow prev = p$; $s \rightarrow next = p \rightarrow next$;C. $s \rightarrow next = p \rightarrow next$; $s \rightarrow prev = p$; $p \rightarrow next \rightarrow prev = s$; $p \rightarrow next = s$;D. $s \rightarrow prev = p$; $s \rightarrow next = p \rightarrow next$; $s \rightarrow prev \rightarrow next = s$; $s \rightarrow next \rightarrow prev = s$;

3.

设栈 S 和队列 Q 初始状态为空，元素 e1, e2, e3, e4, e5, e6 依次通过栈 S，一个元素出栈后即进队列 Q，若 6 个元素出队序列是 e2, e4, e3, e6, e5, e1，则栈 S 的容量至少是 ()

A.2;

B.3;

C.4;

D.6

4.

设循环队列的容量为 40（序号从 0 到 39），现经过一系列的入队和出队运算后：1)

front=12, rear=19; 2) front=19, rear=12; 在这两种情况下，循环队列中的元素个数分别为和。

5.

一个 n 位的字符串共有个子串。

6.

已知二叉树有 n 个结点 ($n > 0$)，则度为 2 的结点最多有个。

7.

用数组存储一个有 50 个元素的最小值堆。已知堆中没有重复元素。给出最大元素的下标可能的取值范围为到。

8.

假设用于通信的电文由 5 个字符组成。已知其中一个字符的 Huffman 编码为 1101，则该 Huffman 编码树的平均编码长度为。

9.

假设先根次序遍历某棵树的结点次序为 GABCDIJEFH，后根次序遍历该树的结点次序为 BADIJCFEHG，那么 I 结点的父结点是。

10. 一个拥有 144 个结点的完全二叉树，现在从倒数第二层上任取一个结点，该结点为叶结点

二、辨析与简答 (共 24 分)

1.

(6 分) 现有一个由单链表和循环单链表构成的特殊链表，单链表的头元素是 head，末尾元素为 tail，head->...->tail 即是一条普通的链表，同时 tail 元素还是另一条循环单链表的头元素。如 A->B->C->D->E->F->G->H->E 就是一个特殊链表，其中 head 为 A，tail 为 E。现在你只知道 head，但是你知道这个循环链表中有多少元素，但同时你的剩余空间十分的小，所以你想用尽可能小的空间来解决问题，请问你会怎么做？（依据占用空间给分）

2.

(6 分) 假设以 I 和 O 分别表示入栈和出栈操作，栈的初始和终态均为空，入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列，称可以操作的序列为合法序列，否则为非法序列。如 IOIOIOIO 合法，IOOIOIO 和 IOOIOIO 为非法。请给出一个算法，判断所给操作序列是否合法。

3.

(6 分) 给定一棵用数组存储的完全二叉树 {10, 5, 12, 3, 2, 1, 8}

(1) 用筛选建堆法将该完全二叉树调整为最大值堆。画出调整后得到的最大值堆，并说明在调整过程中都依次交换了哪些元素。（注：画堆只需要写出层次遍历序列结果即可）

(2) 将 6、7、14 按顺序插入 (1) 得到的最大值堆，堆的空间足够大，画出插入 14 后得到的最大值堆，并说明在插入 14 的过程中都依次交换了哪些元素。

(3) 对 (2) 得到的堆进行 3 次 deleteMax，画出第 3 次 deleteMax 后得到的堆，并说明在第 3 次 deleteMax 的过程中都依次交换了哪些元素。

4.

(6 分) 对下列 15 个等价对进行合并，给出所得等价类树的图示。在初始情况下，集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则，合并时子树结点少的并入结点数多的那棵（多的那个作为新树根，少的那个根作为新根的直接子结点）；若两棵树规模同样大，则把根值较大的并入根值较小（新树根取值小的）。同时，采用路径压缩优化。

(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (12,9) (4,14) (6,7) (8,10) (8,7) (7,11) (10,15) (10,13)

三、算法填空 (每空 3 分，共 24 分)

1.

下面的算法利用一个栈将一给定的序列从小到大排序。长度为 n 的序列由 1, 2, ..., n 这 n 个连续的正整数组成。请补全下面的代码段，使其可以判断给定的序列是否可以利用一个栈进行排序，如果可以，输出相应的操作。

例如：序列 4 3 1 2，此序列可以排序，输出：push push push pop push pop pop pop。

template <class T> // 栈的元素类型为 T

```
class Stack {
```

```
public:
```

```
void clear();
```

```
// 变为空栈
```

```
void push(const T item);
```

```
// item 入栈
```

```
T pop();
```

```
// 返回栈顶内容并弹出
```

```
T top();
```

```
// 返回栈顶内容但不弹出
```

```
bool isEmpty();
```

```
// 若栈已空返回真
```

```
bool isFull();
```

```
// 若栈已满返回真
```

```
};
```

```

Stack<int> s;
// s 为栈
int sequence[maxLen], len; // sequence 为待排序序列，len 为序列长度
bool islegal() {
// 判断序列是否可以排序
int i, j, k;
for (i = 0; i < len; i++) {
for (j = i+1; j < len; j++) {
for (k = j+1; k < len; k++) {
if ( //填空 1 )
return false;
}
}
}
return true;
}
void transform() {
// 输出转换操作
int cur = 1, i = 0;
while ( //填空 2 ) {
while (s.isEmpty() || //填空 3 ) {
//填空 4
cout << "push ";
if (cur == s.top()) break;
}
s.pop();
cout << "pop ";
++cur;
}
}
}

```

2.

下面的算法将一个用带右链的先根次序法表示的森林转换为用带度数的后根次序法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。

```

template<T> // 数据结构
class RlinkTreeNode {
int ltag; // 左标记
T info;
RlinkTreeNode<T> *rlink; // 右链
}
template<T>
class PostTreeNode {
T info;
int degree; // 度数
}
// 算法描述：递归遍历用带右链的先根次序法表示的森林，同时计算出度数并转成后根次序森林；设带右链的先根次序法表示的森林为数组
RlinkTreeNode RlinkTree[n];
// 带度数的后根次序法表示的森林为数组
PostTreeNode PostTree[n];
int count = 0; // 计数器

```

```
int convert(PostTreeNode * node) { // 返回值表示所有右兄弟的数量（包括自己）
int tmp;
if (1 == node->ltag) { // 若没有左子结点，将该结点输出到 PostTree
// 填空 5
PostTree[count].degree = 0; // 度数必然是 0
count++;
}
else { // 有左子结点则压栈
tmp = convert(node+1); // 访问第一个子结点，并返回度数
PostTree[count].info = node->info;
// 填空 6
count++;
}
if ( // 填空 7 ) { // 有右兄弟（，访问并返回右边兄弟的数量 +1）
return convert(node->rlink) + 1;
}
else { // 没有右兄弟
// 填空 8
}
}
```

四、算法设计与实现 (14 分)

1.

(8 分) 编写算法伪码，对给定的字符串 str，返回其最长重复子串及其下标位置。如 str="abcdaccdac"，则子串"cdac"是 str 的最长重复子串，下标为 2。

(重复子串允许重叠)

2.

(6 分) 给出一个算法，求一棵树的树高。可以写出伪代码，需要相应的注释，或者写出详细算法思路。对时间复杂度无具体要求。

五、分析证明题 (共 14 分)

1.

(8 分) 二叉树的内部路径长度是指所有节点的深度的和。假设将 N 个互不相同的随机元素插入一棵空的二叉搜索树。请证明得到的二叉搜索树的内部路径长度期望为 $O(N\log N)$ 。

2.

(6 分) 证明按先根次序周游树获得的序列与其对应二叉树的前序周游获得的序列相同。

数据结构与算法 A

17 秋-期中

题目来源为真题扫描件转录。客观题答案和部分参考解答已整理在解答中；原扫描中考场纪律、装订线等非题目信息已尽量删去。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

1. 一、选择与填空（每空 2 分，共 24 分）

1.

下面函数的时间复杂度是 O

```
def recursive(int n, int m, int k){  
    if (n <= 0)  
        print(m, k)  
    else {  
        recursive(n-1, m+1, k);  
        recursive(n-1, m, k+1);  
    }  
}
```

A. $O(n*m*k)$ B. $O(n^2*m^2)$

B.

$O(2^n)$ D. $O(n!)$

2.

完成在双循环链表结点 p 之后插入 s 的操作为 ():

A. $p.next.prev=s; s.prev=p; s.next=p.next; p.next=s;$

B. $p.next.prev=s; p.next=s; s.prev=p; s.next=p.next;$

C. $s.next=p.next; s.prev=p; p.next.prev=s; p.next=s;$

D. $s.prev=p; s.next=p.next; s.prev.next=s; s.next.prev=s;$

3.

设栈 S 和队列 Q 初始状态为空，元素 e1, e2, e3, e4, e5, e6 依次通过栈 S，一个元素出栈后即进队列 Q，若 6 个元素出队序列是 e2, e4, e3, e6, e5, e1，则栈 S 的容量至少是 ()

A.2;

B.3;

C.4;

D.6

4.

设循环队列的容量为 40（序号从 0 到 39），现经过一系列的入队和出队运算后：1)

front=12, rear=19; 2) front=19, rear=12; 在这两种情况下，循环队列中的元素个数分别为和。

5.

一个 n 位的字符串共有个子串。

6.

已知二叉树有 n 个结点 ($n > 0$)，则度为 2 的结点最多有个。

7.

用数组存储一个有 50 个元素的最小值堆。已知堆中没有重复元素。给出最大元素的下标可能的取值范围为到。

8.

假设用于通信的电文由 5 个字符组成。已知其中一个字符的 Huffman 编码为 1101，则该 Huffman 编码树的平均编码长度为。

9.

假设先根次序遍历某棵树的结点次序为 GABCDIJEFH，后根次序遍历该树的结点次序为 BADIJCFEHG，那么 I 结点的父结点是。

10. 一个拥有 144 个结点的完全二叉树，现在从倒数第二层上任取一个结点，该结点为叶结点

二、辨析与简答 (共 24 分)

1.

(6 分) 现有一个由单链表和循环单链表构成的特殊链表，单链表的头元素是 head，末尾元素为 tail，head...tail 即是一条普通的链表，同时 tail 元素还是另一条循环单链表的头元素。如 A.B.C.D.E.F.G.H.E 就是一个特殊链表，其中 head 为 A，tail 为 E。

现在你只知道 head，但是你知道这个循环链表中有多少元素，但同时你的剩余空间十分的小，所以你想用尽可能小的空间来解决问题，请问你会怎么做？（依据占用空间给分）

2.

(6 分) 假设以 I 和 O 分别表示入栈和出栈操作，栈的初始和终态均为空，入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列，称可以操作的序列为合法序列，否则为非法序列。如 IOIOIOIO 合法，IOOIOIO 和 IOOIOIO 为非法。请给出一个算法，判断所给操作序列是否合法。

3.

(6 分) 给定一棵用数组存储的完全二叉树 {10, 5, 12, 3, 2, 1, 8}

(1) 用筛选建堆法将该完全二叉树调整为最大值堆。画出调整后得到的最大值堆，并说明在调整过程中都依次交换了哪些元素。（注：画堆只需要写出层次遍历序列结果即可）

(2) 将 6、7、14 按顺序插入 (1) 得到的最大值堆，堆的空间足够大，画出插入 14 后得到的最大值堆，并说明在插入 14 的过程中都依次交换了哪些元素。

(3) 对 (2) 得到的堆进行 3 次 deleteMax，画出第 3 次 deleteMax 后得到的堆，并说明在第 3 次 deleteMax 的过程中都依次交换了哪些元素。

4.

(6 分) 对下列 15 个等价对进行合并，给出所得等价类树的图示。在初始情况下，集合中的每个元素分别在独立的等价类中。使用重量权衡合并规则，合并时子树结点少的并入结点数多的那棵（多的那个作为新树根，少的那个根作为新根的直接子结点）；若两棵树规模同样大，则把根值较大的并入根值较小（新树根取值小的）。同时，采用路径压缩优化。

(0,2) (1,2) (3,4) (3,1) (3,5) (9,11) (12,14) (12,9) (4,14) (6,7) (8,10) (8,7) (7,11) (10,15) (10,13)

三、算法填空 (每空 3 分，共 24 分)

1.

下面的算法利用一个栈将一给定的序列从小到大排序。长度为 n 的序列由 1, 2, ..., n 这 n 个连续的正整数组成。请补全下面的代码段，使其可以判断给定的序列是否可以利用一个栈进行排序，如果可以，输出相应的操作。

例如：序列 4 3 1 2，此序列可以排序，输出：push push push pop push pop pop pop。

template <class T> // 栈的元素类型为 T

class Stack {

public:

def clear();

// 变为空栈

def push(const T item);

// item 入栈

T pop();

// 返回栈顶内容并弹出

T top();

// 返回栈顶内容但不弹出

isEmpty();

// 若栈已空返回真

isFull();

// 若栈已满返回真

};

```

Stack<int> s;
// s 为栈
int sequence[maxLen], len; // sequence 为待排序序列，len 为序列长度
islegal() {
// 判断序列是否可以排序
int i, j, k;
for (i = 0; i < len; i++) {
for (j = i+1; j < len; j++) {
for (k = j+1; k < len; k++) {
if ( //填空 1 )
return False;
}
}
}
return True;
}
def transform() {
// 输出转换操作
int cur = 1, i = 0;
while ( //填空 2 ) {
while (s.isEmpty() or //填空 3 ) {
//填空 4
cout << "push ";
if (cur == s.top()) break;
}
s.pop();
print("pop", end=" ")
++cur;
}
}
}
2.

```

下面的算法将一个用带右链的先根次序法表示的森林转换为用带度数的后根次序法表示。请利用题目给出的树结点 ADT 和栈 ADT，填充空格，使其成为完整的算法。

```

template<T> // 数据结构
class RlinkTreeNode {
int ltag; // 左标记
T info;
RlinkTreeNode<T> *rlink; // 右链
}
template<T>
class PostTreeNode {
T info;
int degree; // 度数
}
// 算法描述：递归遍历用带右链的先根次序法表示的森林，同时计算出度数并转成后根次序森林；设带右链的先根次序法表示的森林为数组
RlinkTreeNode RlinkTree[n];
// 带度数的后根次序法表示的森林为数组
PostTreeNode PostTree[n];
int count = 0; // 计数器

```

```
int convert(PostTreeNode * node) { // 返回值表示所有右兄弟的数量（包括自己）
int tmp;
if (1 == node.ltag) { // 若没有左子结点，将该结点输出到 PostTree
// 填空 5
PostTree[count].degree = 0; // 度数必然是 0
count++;
}
else { // 有左子结点则压栈
tmp = convert(node+1); // 访问第一个子结点，并返回度数
PostTree[count].info = node.info;
// 填空 6
count++;
}
if ( // 填空 7 ) { // 有右兄弟（，访问并返回右边兄弟的数量 +1）
return convert(node.rlink) + 1;
}
else { // 没有右兄弟
// 填空 8
}
}
```

四、算法设计与实现 (14 分)

1.

(8 分) 编写算法伪码，对给定的字符串 str，返回其最长重复子串及其下标位置。如 str="abcdaccdac"，则子串"cdac"是 str 的最长重复子串，下标为 2。

(重复子串允许重叠)

2.

(6 分) 给出一个算法，求一棵树的树高。可以写出伪代码，需要相应的注释，或者写出详细算法思路。对时间复杂度无具体要求。

五、分析证明题 (共 14 分)

1.

(8 分) 二叉树的内部路径长度是指所有节点的深度的和。假设将 N 个互不相同的随机元素插入一棵空的二叉搜索树。请证明得到的二叉搜索树的内部路径长度期望为 $O(N\log N)$ 。

2.

(6 分) 证明按先根次序周游树获得的序列与其对应二叉树的前序周游获得的序列相同。

数据结构与算法 A

25 春-样题 2

题目来源：数据结构与算法 A 样题，参考解答由 AI 整理。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

一、选择题（30 分，每小题 2 分）

1. 下列叙述中正确的是（ ）。
 - (1) 散列是一种基于索引的逻辑结构
 - (2) 基于顺序表实现的逻辑结构属于线性结构
 - (3) 数据结构设计影响算法效率，逻辑结构起到了决定作用
 - (4) 一个逻辑结构可以有多种类型的存储结构，且不同类型的存储结构会直接影响到数据处理的效率
2. 在广度优先搜索算法中，一般使用什么辅助数据结构？（ ）
 - (1) 队列
 - (2) 栈
 - (3) 树
 - (4) 散列
3. 若某线性表采取链式存储，那么该线性表中结点的存储地址（ ）。
 - (1) 一定不连续
 - (2) 既可连续亦可不连续
 - (3) 一定连续
 - (4) 与头结点存储地址保持连续
4. 若某线性表常用操作是在表尾插入或删除元素，则时间开销最小的存储方式是（ ）。
 - (1) 单链表
 - (2) 仅有头指针的单循环链表
 - (3) 顺序表
 - (4) 仅有尾指针的单循环链表
5. 给定后缀表达式（逆波兰式） $ab+-c*d-$ 对应的中缀表达式是（ ）。
 - (1) $a - b - c * d$
 - (2) $-(a + b) * c - d$
 - (3) $a + b * c - d$
 - (4) $(a + b) * (-c - d)$
6. 今有一空栈 S ，基于待进栈的数据元素序列 a, b, c, d, e, f 依次进行进栈、进栈、出栈、进栈、进栈、出栈的操作，上述操作完成以后，栈 S 的栈顶元素为（ ）。
 - (1) f
 - (2) c
 - (3) a
 - (4) b
7. 对于单链表，表头节点为 $head$ ，判定空表的条件是（ ）。
 - (1) $head \rightarrow next == NULL$
 - (2) $head != NULL$
 - (3) $head \rightarrow next == head$

(4) `head == NULL`

8. 以下典型排序算法中，具有稳定排序特性的是（ ）。

- (1) 冒泡排序 (Bubble Sort)
- (2) 直接选择排序 (Selection Sort)
- (3) 快速排序 (Quick Sort)
- (4) 希尔排序 (Shell Sort)

9. 以下关键字列表中，可以有效构成一个大根堆的序列是（ ）。

- (1) 5 8 1 3 9 6 2 7
- (2) 9 8 1 7 5 6 2 33
- (3) 9 8 6 3 5 1 2 7
- (4) 9 8 6 7 5 1 2 3

10. 以下典型排序算法中，内存开销（包括时间和空间）最大的是（ ）。

- (1) 冒泡排序 (时间 $O(n^2)$, 空间 $O(1)$)
- (2) 快速排序 (时间 $O(n \log n)$, 空间 $O(\log n)$)
- (3) 归并排序 (时间 $O(n \log n)$, 空间 $O(n)$)
- (4) 堆排序 (时间 $O(n \log n)$, 空间 $O(1)$)

11. 排序算法依赖于对元素序列的多趟比较/移动操作，第一趟结束后，任一元素都无法确定其最终排序位置的算法是（ ）。

- (1) 选择排序
- (2) 快速排序
- (3) 冒泡排序
- (4) 插入排序

12. 考察以下基于单链表的操作，相较于顺序表实现，带来更高时间复杂度的操作是（ ）。

- (1) 合并两个有序线性表，并保持合成后的线性表依然有序
- (2) 交换第一个元素与第二个元素的值
- (3) 查找某一元素值是否在线性表中出现
- (4) 输出第 i 个 ($0 \leq i < n$) 元素

13. 已知一个整型数组序列 {19, 20, 50, 61, 73, 85, 11, 39}，采用某种排序算法，多趟比较后的中间结果如下：

19 20 11 39 73 85 50 61
11 20 19 39 50 61 73 85
11 19 20 39 50 61 73 85

请问上述过程使用的排序算法是（ ）。

- (1) 冒泡排序
- (2) 插入排序
- (3) 希尔排序
- (4) 归并排序

14. 今有一非连通无向图，共有 36 条边，该图至少有（ ）个顶点。

- (1) 8
- (2) 9
- (3) 10

(4) 11

15. 令 $G = (V, E)$ 是一个无向图, 若 G 中任何两个顶点之间均存在唯一的简单路径相连, 则下面说法中错误的是 ()。

- (1) 图 G 中添加任何一条边, 不一定造成图包含一个环
- (2) 图 G 中移除任意一条边得到的图均不连通
- (3) 图 G 的逻辑结构实际上退化为树结构
- (4) 图 G 中边的数目一定等于顶点数目减 1

二、判断题（10 分，每小题 1 分；对填 Y，错填 N）

16. 按照前序、中序、后序方式周游一棵二叉树，分别得到不同的结点周游序列，然而三种不同的周游序列中，叶子结点都将以相同的顺序出现。（ ）
17. 构建一个含 N 个结点的（二叉）最小值堆，时间效率最优情况下的时间复杂度为 $O(N \log N)$ 。（ ）
18. 对任意一个连通的无向图，如果存在一个环，且这个环中的一条边的权值不小于该环中任意一个其它的边的权值，那么这条边一定不会是该无向图的最小生成树中的边。（ ）
19. 通过树的周游可以求得树的高度，若采取深度优先遍历方式设计求解树高度问题的算法，算法空间复杂度为 $O(\text{树的高度})$ 。（ ）
20. 树可以等价转化二叉树，树的先序遍历序列与其相应的二叉树的前序遍历序列相同。（ ）
21. 如果一个连通无向图 G 中所有边的权值均不同，则 G 具有唯一的最小生成树。（ ）
22. 求解最小生成树问题的 Prim 算法是一种贪心算法。（ ）
23. 使用线性探测法处理散列表碰撞问题，若表中仍有空槽（空单元），插入操作一定成功。（ ）
24. 从链表中删除某个指定值的结点，其时间复杂度是 $O(1)$ 。（ ）
25. Dijkstra 算法的局限性是无法正确求解带有负权值边的图的最短路径。（ ）

三、填空题（20 分，每空 2 分）

26. 定义二叉树中一个结点的度数为其子结点的个数。现有一棵结点总数为 101 的二叉树，其中度数为 1 的结点数有 30 个，则度数为 0 的结点有 _____ 个。
27. 定义完全二叉树的根结点所在层为第一层。如果一个二叉树的第六层有 23 个叶结点，则它的总结点数可能为 _____。
28. 对于初始排序码序列 (51, 41, 31, 21, 61, 71, 81, 11, 91)，第 1 趟快速排序（以第一个数字为中值）的结果是：_____。
29. 如果输入序列是已经正序，在冒泡排序、直接插入排序和直接选择排序中，_____ 算法最慢结束。
30. 已知某二叉树的先根周游序列为 {A, B, D, E, C, F, G}，中根周游序列为 {D, B, E, A, C, G, F}，则该二叉树的后根次序周游序列为 _____。
31. 使用栈计算后缀表达式（操作数均为一位数）1 2 3 + 4 * 5 + 3 + -，当扫描到第二个 + 号但还未对该 + 号进行运算时，栈的内容（栈底到栈顶从左往右）为：_____。
32. 51 个顶点的连通图 G 有 50 条边，其中权值为 1 至 10 的边各 5 条，则连通图 G 的最小生成树各边的权值之和为 _____。
33. 包含 n 个顶点无向图的邻接表存储结构中，所有顶点的边表中最多有 _____ 个结点。具有 n 个顶点的有向图，每个顶点入度和出度之和最大值不超过 _____。
34. 给定一个长度为 7 的空散列表，采用双散列法解决冲突，两个散列函数分别为： $h_1(\text{key}) = \text{key} \% 7$ ， $h_2(\text{key}) = \text{key} \% 5 + 1$ 。请向散列表依次插入关键字 30, 58, 65，插入完成后 65 在散列表中存储地址为 _____。
35. 阅读算法 ABC(n)，回答问题。

```
int ABC(int n){
    int k = 2, m = (int)sqrt(n);
    while (k <= m && n % k != 0)
        k++;
    return k > m;
}
```

算法的功能是：_____。算法的时间复杂度是 $O(\text{_____})$ 。

四、简答题（共 14 分）

36. (字符串匹配)（4 分）

字符串匹配算法从长度为 n 的文本串 S 中查找长度为 m 的模式串 P 的首次出现。

- (1) 朴素算法时间复杂度为 $O((n - m + 1)m)$ 。请举例说明算法时间复杂度的一种最坏情况（例子中只出现 a 和 b 两种字符）。(1 分)
- (2) 已知 $S = \text{"abaabaabaabcc"}$, $P = \text{"abaabc"}$, 采用朴素算法进行查找, 请写出字符比对的总次数和查找结果。(2 分)
- (3) 朴素算法存在很大的改进空间。说明在 (b) 中第一次出现不匹配 ($s[i + j] \neq t[j]$, $i = 0, j = 5$) 时, 为了避免冗余比对, 下次比对时 i 和 j 的值可以分别调整为多少?(1 分)

37. (AOV 网络与拓扑排序)（5 分）

有八项活动 $V_0 \sim V_7$, 每项活动要求的前驱如下:

活动	前驱
V_0	无
V_1	V_0
V_2	V_0
V_3	V_0, V_2
V_4	V_1
V_5	V_2, V_4
V_6	V_3
V_7	V_5, V_6

- (1) 画出相应的 AOV 网络（顶点为活动，边为先后关系的有向图）；
- (2) 给出一个拓扑排序序列。若存在多种，按编号从小到大排序，输出最小的一种。

38. (二叉查找树)（5 分）

简要回答下列 BST 树以及 BST 树更新过程的相关问题。

- (1) 请简述什么是二叉查找树（BST）。(1 分)
- (2) 请图示 2, 1, 6, 4, 5, 3 按顺序插入一棵 BST 树的中间过程和最终形态。(2 分)
- (3) 请图示以上 BST 树依次删除节点 4 和 2 的过程和树的形态。(2 分)

五、算法填空（共 26 分）

39. (拓扑排序)（6 分）

给定一个有向图，求拓扑排序序列。输入：第一行是整数 n ($1 \leq n \leq 100$)，接下来 n 行，第 i 行列出了顶点 i 的所有邻点，以 0 结尾。输出：一个拓扑排序序列。如果图中有环，则输出 "Loop"。

```
typedef struct Edge { int v; struct Edge *next; } Edge;
int topoSort(Graph *G, vector<int> &seq){
    int n = G->n;
    Queue q; initQueue(&q);
    int inDegree[MAXV] = {0};
    for (int i = 0; i < n; ++i)
        for (Edge *e = G->adj[i]; e != NULL; e = e->next)
            _____ // (1) 1 分
    for (int i = 0; i < n; ++i)
        if (inDegree[i] == 0)
            _____ // (2) 1 分
    while (!empty(&q)) {
        int k = dequeue(&q);
        _____ // (3) 1 分
        for (Edge *e = G->adj[k]; e != NULL; e = e->next) {
            _____ // (4) 1 分
            if (inDegree[e->v] == 0)
                _____ // (5) 1 分
        }
    }
    if (_____) // (6) 1 分
        return 0;
    else
        return 1;
}
```

40. (链表去重)（7 分）

读入一个从小到大排好序的整数序列到链表，然后在链表中删除重复的元素，使得重复的元素只保留 1 个，然后将整个链表内容输出。

```
typedef struct Node { int data; struct Node *next; } Node;
/* 读入数组 a[0..n-1] */
Node *head = newNode(a[0]);
Node *p = head;
for (int i = 1; i < n; ++i) {
    _____ // (1) 2 分
    p = p->next;
}
p = head;
while (p != NULL) {
    while (p->next != NULL && p->data == p->next->data)
        _____ // (2) 3 分
    _____ // (3) 2 分
}
p = head;
while (p != NULL) {
    printf("%d ", p->data);
    p = p->next;
}
```

41. (无向图连通性与回路判定)（7 分）

给定一个无向图，判断是否连通，是否有回路。

```

bool isConnected(Graph *G){
    int total = 0;
    bool visited[MAXV] = {false};
    dfsCount(G, 0, visited, &total);
    _____ // (1) 2 分
}

bool dfsCycle(Graph *G, int v, int parent, bool visited[]){
    visited[v] = true;
    for (Edge *e = G->adj[v]; e != NULL; e = e->next) {
        int u = e->v;
        if (visited[u]) {
            _____ // (2) 2 分
            return true;
        } else {
            _____ // (3) 2 分
            return true;
        }
    }
    return false;
}

bool hasLoop(Graph *G){
    bool visited[MAXV] = {false};
    for (int i = 0; i < G->n; ++i)
        _____ // (4) 1 分
    if (dfsCycle(G, i, -1, visited)) return true;
    return false;
}

```

42. (堆排序) (6 分)

输入若干个整数，使用堆排序算法从小到大排序（建立大顶堆）。

```

void heapSort(int a[], int n){
    int heapSize = n;
    void goDown(int i){
        int L = i * 2 + 1, R = i * 2 + 2, s;
        if (L >= heapSize) return;
        _____ // (1) 1 分
        if (a[s] > a[i]) {
            _____ // (2) 2 分
            goDown(s);
        }
    }
    _____ // (3) 1 分
    for (int i = n - 1; i >= 0; --i) {
        _____ // (4) 1 分
        heapSize--;
        _____ // (5) 1 分
    }
}

```

数据结构与算法 A

25 春-样题 2

题目来源：数据结构与算法 A 样题，参考解答由 AI 整理。本次整理提供 C 语言版与 Python 语言版；除程序代码语言外，两版题目内容保持一致。

一、选择题（30 分，每小题 2 分）

- 下列叙述中正确的是（ ）。
 - 散列是一种基于索引的逻辑结构
 - 基于顺序表实现的逻辑结构属于线性结构
 - 数据结构设计影响算法效率，逻辑结构起到了决定作用
 - 一个逻辑结构可以有多种类型的存储结构，且不同类型的存储结构会直接影响到数据处理的效率
- 在广度优先搜索算法中，一般使用什么辅助数据结构？（ ）
 - 队列
 - 栈
 - 树
 - 散列
- 若某线性表采取链式存储，那么该线性表中结点的存储地址（ ）。
 - 一定不连续
 - 既可连续亦可不连续
 - 一定连续
 - 与头结点存储地址保持连续
- 若某线性表常用操作是在表尾插入或删除元素，则时间开销最小的存储方式是（ ）。
 - 单链表
 - 仅有头指针的单循环链表
 - 顺序表
 - 仅有尾指针的单循环链表
- 给定后缀表达式（逆波兰式） $ab+-c*d-$ 对应的中缀表达式是（ ）。
 - $a - b - c * d$
 - $-(a + b) * c - d$
 - $a + b * c - d$
 - $(a + b) * (-c - d)$
- 今有一空栈 S ，基于待进栈的数据元素序列 a, b, c, d, e, f 依次进行进栈、进栈、出栈、进栈、进栈、出栈的操作，上述操作完成以后，栈 S 的栈顶元素为（ ）。
 - f
 - c
 - a
 - b
- 对于单链表，表头节点为 $head$ ，判定空表的条件是（ ）。
 - $head.next == None$
 - $head != None$
 - $head.next == head$

(4) `head == None`

8. 以下典型排序算法中，具有稳定排序特性的是（ ）。

- (1) 冒泡排序 (Bubble Sort)
- (2) 直接选择排序 (Selection Sort)
- (3) 快速排序 (Quick Sort)
- (4) 希尔排序 (Shell Sort)

9. 以下关键字列表中，可以有效构成一个大根堆的序列是（ ）。

- (1) 5 8 1 3 9 6 2 7
- (2) 9 8 1 7 5 6 2 33
- (3) 9 8 6 3 5 1 2 7
- (4) 9 8 6 7 5 1 2 3

10. 以下典型排序算法中，内存开销（包括时间和空间）最大的是（ ）。

- (1) 冒泡排序（时间 $O(n^2)$ ，空间 $O(1)$ ）
- (2) 快速排序（时间 $O(n \log n)$ ，空间 $O(\log n)$ ）
- (3) 归并排序（时间 $O(n \log n)$ ，空间 $O(n)$ ）
- (4) 堆排序（时间 $O(n \log n)$ ，空间 $O(1)$ ）

11. 排序算法依赖于对元素序列的多趟比较/移动操作，第一趟结束后，任一元素都无法确定其最终排序位置的算法是（ ）。

- (1) 选择排序
- (2) 快速排序
- (3) 冒泡排序
- (4) 插入排序

12. 考察以下基于单链表的操作，相较于顺序表实现，带来更高时间复杂度的操作是（ ）。

- (1) 合并两个有序线性表，并保持合成后的线性表依然有序
- (2) 交换第一个元素与第二个元素的值
- (3) 查找某一元素值是否在线性表中出现
- (4) 输出第 i 个 ($0 \leq i < n$) 元素

13. 已知一个整型数组序列 {19, 20, 50, 61, 73, 85, 11, 39}，采用某种排序算法，多趟比较后的中间结果如下：

19 20 11 39 73 85 50 61
11 20 19 39 50 61 73 85
11 19 20 39 50 61 73 85

请问上述过程使用的排序算法是（ ）。

- (1) 冒泡排序
- (2) 插入排序
- (3) 希尔排序
- (4) 归并排序

14. 今有一非连通无向图，共有 36 条边，该图至少有（ ）个顶点。

- (1) 8
- (2) 9
- (3) 10

(4) 11

15. 令 $G = (V, E)$ 是一个无向图, 若 G 中任何两个顶点之间均存在唯一的简单路径相连, 则下面说法中错误的是 ()。

- (1) 图 G 中添加任何一条边, 不一定造成图包含一个环
- (2) 图 G 中移除任意一条边得到的图均不连通
- (3) 图 G 的逻辑结构实际上退化为树结构
- (4) 图 G 中边的数目一定等于顶点数目减 1

二、判断题（10 分，每小题 1 分；对填 Y，错填 N）

16. 按照前序、中序、后序方式周游一棵二叉树，分别得到不同的结点周游序列，然而三种不同的周游序列中，叶子结点都将以相同的顺序出现。（ ）
17. 构建一个含 N 个结点的（二叉）最小值堆，时间效率最优情况下的时间复杂度为 $O(N \log N)$ 。（ ）
18. 对任意一个连通的无向图，如果存在一个环，且这个环中的一条边的权值不小于该环中任意一个其它的边的权值，那么这条边一定不会是该无向图的最小生成树中的边。（ ）
19. 通过树的周游可以求得树的高度，若采取深度优先遍历方式设计求解树高度问题的算法，算法空间复杂度为 $O(\text{树的高度})$ 。（ ）
20. 树可以等价转化二叉树，树的先序遍历序列与其相应的二叉树的前序遍历序列相同。（ ）
21. 如果一个连通无向图 G 中所有边的权值均不同，则 G 具有唯一的最小生成树。（ ）
22. 求解最小生成树问题的 Prim 算法是一种贪心算法。（ ）
23. 使用线性探测法处理散列表碰撞问题，若表中仍有空槽（空单元），插入操作一定成功。（ ）
24. 从链表中删除某个指定值的结点，其时间复杂度是 $O(1)$ 。（ ）
25. Dijkstra 算法的局限性是无法正确求解带有负权值边的图的最短路径。（ ）

三、填空题（20 分，每空 2 分）

26. 定义二叉树中一个结点的度数为其子结点的个数。现有一棵结点总数为 101 的二叉树，其中度数为 1 的结点数有 30 个，则度数为 0 的结点有 _____ 个。
27. 定义完全二叉树的根结点所在层为第一层。如果一个二叉树的第六层有 23 个叶结点，则它的总结点数可能为 _____。
28. 对于初始排序码序列 (51, 41, 31, 21, 61, 71, 81, 11, 91)，第 1 趟快速排序（以第一个数字为中值）的结果是：_____。
29. 如果输入序列是已经正序，在冒泡排序、直接插入排序和直接选择排序中，_____ 算法最慢结束。
30. 已知某二叉树的先根周游序列为 {A, B, D, E, C, F, G}，中根周游序列为 {D, B, E, A, C, G, F}，则该二叉树的后根次序周游序列为 _____。
31. 使用栈计算后缀表达式（操作数均为一位数）1 2 3 + 4 * 5 + 3 + -，当扫描到第二个 + 号但还未对该 + 号进行运算时，栈的内容（栈底到栈顶从左往右）为：_____。
32. 51 个顶点的连通图 G 有 50 条边，其中权值为 1 至 10 的边各 5 条，则连通图 G 的最小生成树各边的权值之和为 _____。
33. 包含 n 个顶点无向图的邻接表存储结构中，所有顶点的边表中最多有 _____ 个结点。具有 n 个顶点的有向图，每个顶点入度和出度之和最大值不超过 _____。
34. 给定一个长度为 7 的空散列表，采用双散列法解决冲突，两个散列函数分别为： $h_1(\text{key}) = \text{key} \% 7$ ， $h_2(\text{key}) = \text{key} \% 5 + 1$ 。请向散列表依次插入关键字 30, 58, 65，插入完成后 65 在散列表中存储地址为 _____。
35. 阅读算法 ABC(n)，回答问题。

```
def ABC(n):
    k, m = 2, int(n**0.5)
    while (k <= m) and (n % k != 0):
        k += 1
    return k > m
```

算法的功能是：_____。算法的时间复杂度是 $O(\text{_____})$ 。

四、简答题（共 14 分）

36. (字符串匹配)（4 分）

字符串匹配算法从长度为 n 的文本串 S 中查找长度为 m 的模式串 P 的首次出现。

- (1) 朴素算法时间复杂度为 $O((n - m + 1)m)$ 。请举例说明算法时间复杂度的一种最坏情况（例子中只出现 a 和 b 两种字符）。(1 分)
- (2) 已知 $S = \text{"abaabaabaabcc"}$, $P = \text{"abaabc"}$, 采用朴素算法进行查找, 请写出字符比对的总次数和查找结果。(2 分)
- (3) 朴素算法存在很大的改进空间。说明在 (b) 中第一次出现不匹配 ($s[i + j] \neq t[j]$, $i = 0, j = 5$) 时, 为了避免冗余比对, 下次比对时 i 和 j 的值可以分别调整为多少?(1 分)

37. (AOV 网络与拓扑排序)（5 分）

有八项活动 $V_0 \sim V_7$, 每项活动要求的前驱如下:

活动	前驱
V_0	无
V_1	V_0
V_2	V_0
V_3	V_0, V_2
V_4	V_1
V_5	V_2, V_4
V_6	V_3
V_7	V_5, V_6

- (1) 画出相应的 AOV 网络（顶点为活动，边为先后关系的有向图）；
- (2) 给出一个拓扑排序序列。若存在多种，按编号从小到大排序，输出最小的一种。

38. (二叉查找树)（5 分）

简要回答下列 BST 树以及 BST 树更新过程的相关问题。

- (1) 请简述什么是二叉查找树（BST）。(1 分)
- (2) 请图示 2, 1, 6, 4, 5, 3 按顺序插入一棵 BST 树的中间过程和最终形态。(2 分)
- (3) 请图示以上 BST 树依次删除节点 4 和 2 的过程和树的形态。(2 分)

五、算法填空（共 26 分）

39. (拓扑排序)（6 分）

给定一个有向图，求拓扑排序序列。输入：第一行是整数 n ($1 \leq n \leq 100$)，接下来 n 行，第 i 行列出了顶点 i 的所有邻点，以 0 结尾。输出：一个拓扑排序序列。如果图中有环，则输出 "Loop"。

```
class Edge:
    def __init__(self, v):
        self.v = v

def topoSort(G):
    n = len(G)
    import queue
    inDegree = [0] * n
    q = queue.Queue()
    for i in range(n):
        for e in G[i]:
            _____ # (1) 1 分
        for i in range(n):
            if inDegree[i] == 0:
                _____ # (2) 1 分
            seq = []
            while not q.empty():
                k = q.get()
                _____ # (3) 1 分
            for e in G[k]:
                _____ # (4) 1 分
            if inDegree[e.v] == 0:
                _____ # (5) 1 分
            if _____: # (6) 1 分
                return None
            else:
                return seq
```

40. (链表去重)（7 分）

读入一个从小到大排好序的整数序列到链表，然后在链表中删除重复的元素，使得重复的元素只保留 1 个，然后将整个链表内容输出。

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

a = list(map(int, input().split()))
head = Node(a[0])
p = head
for x in a[1:]:
    _____ # (1) 2 分
    p = p.next
    p = head
    while p:
        while p.next and p.data == p.next.data:
            _____ # (2) 1 分 (3) 2 分
            _____ # (4) 2 分
        p = head
    while p:
        print(p.data, end=" ")
        p = p.next
```

41. (无向图连通性与回路判定)（7 分）

给定一个无向图，判断是否连通，是否有回路。

```
def isConnected(G):
    n = len(G)
    visited = [False for i in range(n)]
    total = 0
    def dfs(v):
        nonlocal total
        visited[v] = True
        total += 1
        for u in G[v]:
            if not visited[u]:
                dfs(u)
    dfs(0)
    _____ # (1) 2 分
```

```
def hasLoop(G):
    n = len(G)
    visited = [False for i in range(n)]
    def dfs(v, x):
        visited[v] = True
        for u in G[v]:
            if visited[u] == True:
                _____ # (2) 2 分
            return True
        else:
            _____ # (3) 2 分
        return True
    return False
    for i in range(n):
        _____ # (4) 1 分
    if dfs(i, -1):
        return True
    return False
```

42. (堆排序) (6 分)

输入若干个整数，使用堆排序算法从小到大排序（建立大顶堆）。

```
def heapSort(a):
    heapSize = len(a)
    def goDown(i):
        if i * 2 + 1 >= heapSize:
            return
        L, R = i * 2 + 1, i * 2 + 2
        _____ # (1) 1 分
        s = L
        else:
            s = R
        if a[s] > a[i]:
            _____ # (2) 2 分
        goDown(s)
    def heapify():
        _____ # (3) 1 分
    goDown(k)
    heapify()
    for i in range(len(a) - 1, -1, -1):
        _____ # (4) 1 分
    heapSize -= 1
    _____ # (5) 1 分
```

数据结构与算法 A

课程说明

以下说明依据本项目现有真题整理，主要反映题库中稳定出现的范围。

一、资料概况

本项目当前收录本课程题目若干套，并为每套试卷提供 C 语言版与 Python 版。题目来源包括考试回忆、扫描版真题与样题。

二、考试范围（按现有题库归纳）

- **线性结构：**顺序表、链表、栈、队列的存储与操作；
- **树与二叉树：**遍历、二叉搜索树、堆、平衡树、Huffman 树；
- **图：**存储（邻接矩阵/表）、遍历（BFS/DFS）、最短路、最小生成树、拓扑排序；
- **查找：**顺序/二分查找、二叉搜索树、散列（哈希）；
- **排序：**插入、选择、冒泡、快速、归并、堆、希尔排序及其稳定性与复杂度；
- **算法设计与分析：**递归、分治、贪心、动态规划、复杂度分析。

三、试卷结构

- **选择题：**考察基本概念与简单计算，每题 2 分，约 15 题；
- **简答/计算题：**树/图/排序的手工过程，复杂度分析；
- **算法设计/编程题：**伪代码或 C/Python 代码填空与实现。

四、文件说明

本目录下源文件命名规则：extttt 数据结构与算法 A-时间-期中/期末/样题-C/P，其中 C/P 代表使用 C 语言或 Python 语言。扫描版 PDF 为原始试卷；对应的 .tex 文件为转录排版后的版本。C/P 两版除程序代码语言外，题目内容保持一致。